Y4543102

学 校 代 码 10459
学号或申请号 202222172013495
密 级

郑州大学

专业硕士学位论文

面向国产加速器的 MLIR 卷积算子生成研究

作 者 姓 名：汤文浩
导 师 姓 名：曹武迪 高级工程师
专业学位名称：计算机技术
培 养 院 系：计算机与人工智能学院
完 成 时 间：2025 年 5 月

万方数据

Y4543102

A thesis submitted to
Zhengzhou University
for the degree of Master

Research on MLIR convolution operator generation Based on Domestic Accelerators

By Wenhao Tang
Supervisor: Wudi Cao
Computer Technology
School of Computer and Artificial Intelligence
May,2025

万方数据

学位论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的科研成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本声明的法律责任由本人承担。

学位论文作者签名：汤文浩　　导师签名：[illegible]

日期：　2025 年 5 月 21 日

学位论文使用授权声明

本人在导师指导下完成的论文及相关的职务作品，知识产权归属郑州大学。根据郑州大学有关保留、使用学位论文的规定，同意学校保留或向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权郑州大学可以将本学位论文的全部或部分编入有关数据库进行检索，可以采用影印、缩印或者其他复制手段保存论文和汇编本学位论文。本人离校后发表、使用学位论文或与该学位论文直接相关的学术论文或成果时，第一署名单位仍然为郑州大学。保密论文在解密后应遵守此规定。

学位论文作者签名：汤文浩　　导师签名：[illegible]

日期：　2025 年 5 月 21 日

万方数据

摘要

近年来卷积神经网络在深度学习中愈发重要，在图像分类、目标检测、语义分割、视频分析等方向上都有相关应用。卷积运算是卷积神经网络中的核心计算单元，直接影响网络的计算效率。然而，卷积神经网络涉及大量不同参数的卷积运算，对每组参数进行手动优化不仅成本高昂，而且难以适应多变的硬件环境。此外，随着深度学习框架的不断演进，手写卷积算子需要频繁调整迭代。为解决上述手写卷积算子的问题，本文设计并实现一个适配国产加速器的MLIR(Multi-Level Intermediate Representation)卷积算子生成器。该生成器旨在结合MLIR的优势与国产加速器的特性，实现卷积算子的自动化生成，降低开发成本。本文的主要工作如下：

(1) 基于MLIR构建了一套分层式的卷积算子生成框架。在框架中，采用分层的下降通路，实现了不同粒度的优化。该框架能够在高层进行算法建模，并将高层的计算表达逐步转换为底层硬件可识别并高效执行的指令序列，从而最大限度地发挥硬件的并行计算能力。

(2) 基于Implicit GEMM(Implicit General Matrix Multiplication)算法，设计了Conv Dialect，以实现卷积在国产加速器上的计算逻辑。在设计过程中，Conv Dialect充分考虑了卷积计算的复杂性和层次结构，并引入了多种Operation设计，以层次化方式描述卷积计算任务。这种分层式设计不仅有助于在不同计算阶段对计算进行优化，还便于将高层计算模型逐步映射为低层硬件执行指令。

(3) 基于矩阵外积、双缓冲流水并行策略等优化算法并利用MMAC(Matrix Multiply-Accumulate)底层指令，构建高效的优化通路。针对矩阵内积在大规模数据时产生的性能瓶颈问题，引入矩阵外积算法，以减少内存访问次数，避免对同一数据的重复加载，从而降低访存开销；为缓解串行执行机制下的访存延迟，提出双缓冲流水并行策略，充分利用国产加速器访存指令与计算指令可并行执行的特性，使数据加载与计算过程重叠，有效隐藏部分延迟。在整个运算过程中使用循环展开技术，减少循环次数，提高指令并行性，从而形成流水并行；为了优化FMA(Fused Multiply-Add)指令在处理大矩阵时的频繁使用所导致的计算瓶颈问题，引入MMAC指令对底层矩阵乘法进行优化。MMAC指令能够在一个周期内完成多个乘加操作，相较于传统的FMA指令，其并行度更高，计算效率更优。

万方数据

通过合理调度MMAC指令，使其充分利用计算单元的吞吐能力，减少数据依赖，提高计算并发度。

最后，针对卷积算子生成器的前向卷积和反向数据卷积，在国产加速器上进行了性能测试。实验设定了两组对照实验：第一组以国产加速器自研手写算子作为基准，第二组选取NVIDIA V100 平台上的cuDNN库算子进行对比，并以能效比作为跨平台性能评估指标。实验结果显示，在前向卷积测试中，生成器在FP32精度下的性能达到国产手写算子库的 85%，在FP16 精度下达到 70%。与V100 平台算子相比，FP32 精度下 80%的算子能效比超过 1，FP16 精度下 73%的算子能效比超过 1。在反向数据卷积测试中，生成器在FP32 精度下的性能达到国产手写算子库的 82%，而与V100 平台算子的能效比整体保持在 77%。

关键词： 国产加速器 MLIR 卷积算子生成器 Implicit GEMM 能效比

万方数据

Abstract

In recent years, convolutional neural networks have become increasingly important in deep learning, with applications in image classification, object detection, semantic segmentation, video analysis, and other areas. Convolution operations are the core computational units in convolutional neural networks, directly affecting the network's computational efficiency and performance. However, there are a large number of convolution operations with different parameters in these networks, and manually optimizing each set of parameters is too costly. In addition, with the rapid iteration of hardware platforms and the continuous evolution of deep learning frameworks, convolution operators require multiple rounds of manual optimization. To address the issue of handwritten convolution operators, this paper designs and implements an MLIR(Multi-Level Intermediate Representation) convolution operator generator adapted to domestic accelerators, aiming to automatically generate convolution operators by combining the advantages of MLIR with the characteristics of domestic accelerators. The main contributions of this paper are as follows:

(1) A hierarchical convolution operator generation framework based on MLIR has been constructed. Within this framework, a reasonable lowering path is adopted to achieve optimizations at different granularities. The framework is capable of performing high-level algorithm modeling and gradually translating high-level computational expressions into instruction sequences that the underlying hardware can recognize and execute efficiently, thereby maximizing the parallel computing capabilities and hardware resources of the GPU.

(2) Based on the Implicit GEMM algorithm, a Conv Dialect has been designed to implement the computational logic of convolution on domestic accelerators. During the design process, the Conv Dialect fully considers the complexity and hierarchical structure of convolution computations and introduces various Operation designs to describe the convolution tasks in a layered manner. This hierarchical design not only facilitates optimizations at different computational stages but also makes it easier to gradually map high-level computational models to low-level hardware execution

万方数据

instructions.

(3) By leveraging matrix outer product optimization, double-buffered pipelined parallel strategies, and MMAC(Matrix Multiply-Accumulate)instructions, an efficient optimization pathway is constructed. The matrix outer product reduces memory access and redundant data loading, lowering overhead and improving efficiency. To mitigate memory latency in serial execution, a loop pipelining strategy is introduced, utilizing domestic accelerators' ability to execute memory and computation instructions in parallel. Double-buffering overlaps data loading and computation, effectively hiding latency, while loop unrolling reduces iterations and enhances instruction parallelism. To address performance bottlenecks caused by frequent FMA(Fused Multiply-Add)operations in large matrix computations, MMAC instructions are introduced to optimize matrix multiplication. These instructions perform multiple multiply-accumulate operations per cycle, improving parallelism, maximizing computational unit throughput, and increasing overall efficiency.

Finally, performance tests were conducted on the forward convolution and backward data convolution of the convolution operator generator on the domestic accelerator. Two sets of comparative experiments were designed: the first set used the manually optimized operators from the domestic accelerator as the baseline, while the second set compared against cuDNN library operators on the NVIDIA V100 platform, with energy efficiency ratio used as the cross-platform performance evaluation metric. The experimental results show that in forward convolution tests, the generator achieved 85% of the performance of the domestic handwritten operator library in FP32 precision and 70% in FP16 precision. Compared to the V100 platform operators, 80% of the operators had an energy efficiency ratio greater than 1 in FP32 precision, while 73% exceeded 1 in FP16 precision. In backward data convolution tests, the generator achieved 82% of the performance of the domestic handwritten operator library in FP32 precision, while maintaining an overall energy efficiency ratio of 77% compared to the V100 platform operators.

Keywords: Domestic Accelerator; MLIR; Convolution Generator; Implicit GEMM; Energy Efficiency Ratio

万方数据

目录

摘要 I
Abstract III
目录 V
图清单 VIII
表清单 IX
1 绪论 1
1.1 研究背景及意义 1
1.2 国内外研究现状 2
1.2.1 MLIR 研究现状 2
1.2.2 国内外加速器现状 4
1.3 研究内容 5
1.4 论文组织架构 6
2 相关理论与技术基础 8
2.1 MLIR 技术理论基础 8
2.1.1 MLIR 概述 8
2.1.2 MLIR Dialect 介绍 9
2.1.3 MLIR Pass 概述 11
2.1.4 MLIR Operation 概述 11
2.2 卷积加速算法 12
2.2.1 im2col + GEMM 算法 12
2.2.2 Implicit GEMM 算法 14
2.3 国产加速器 15
2.3.1 加速器架构 15
2.3.2 存储模型 17
2.3.3 线程模型 18
2.4 本章小节 20

万方数据

3 基于 MLIR 的卷积算子生成器的框架设计 21
3.1 设计理念 21
3.2 整体架构 21
3.2.1 算子生成层 22
3.2.2 建模层 23
3.2.3 实现层 23
3.2.4 执行层 24
3.3 CONV DIALECT 设计 24
3.3.1 软件抽象 25
3.3.2 硬件抽象 25
3.3.3 计算组件 26
3.4 下降通路 27
3.4.1 Conv Dialect 内的 OP 下降 27
3.4.2 Conv Dialect 下降 30
3.5 本章小结 32
4 基于优化算法和底层指令的优化通路设计 33
4.1 矩阵外积 33
4.2 双缓冲流水并行 34
4.2.1 串行执行机制 34
4.2.2 双缓冲流水并行 35
4.3 MMAC 指令 38
4.4 本章小结 39
5 实验结果与性能评估 41
5.1 实验环境及测试用例 41
5.2 实验校验标准 42
5.3 前向卷积实验 43
5.3.1 Fp32 测试 43
5.3.2 Fp16 测试 44
5.4 反向数据卷积实验 46
5.5 本章小结 48

万方数据

6 总结与展望 49
6.1 工作总结 49
6.2 展望 51
参考文献 52
个人简历、在学期间发表的学术论文与研究成果 56
致谢 57

万方数据

图清单

图 2.1 MLIR 的 Dialect 转换 9
图 2.2 Dialect 结构 9
图 2.3 Operation 格式 12
图 2.4 im2col 算法 13
图 2.5 国产类 GPU 加速器架构 16
图 2.6 线程模型 19
图 3.1 mlirGen 架构图 22
图 3.2 Conv Dialect 结构图 24
图 3.3 Conv Dialect 内的 Op 下降 28
图 3.4 TransformOp 下降 29
图 3.5 GemmOp 下降 30
图 4.1 传统矩阵乘内积形式 33
图 4.2 矩阵乘外积计算方式 34
图 4.3 单次循环内线程执行流程 35
图 4.4 双 LDS 示意图 36
图 4.5 线程级循环流水并行 37
图 4.6 mmac_16x16x4_f32 计算过程 39
图 5.1 FP32 精度下 mlirGen 与官方算子库前向卷积性能对比 43
图 5.2 FP32 精度下 mlirGen 与 V100 平台前向卷积算子能效比 44
图 5.3 FP16 精度下 mlirGen 与官方算子库前向卷积性能对比 45
图 5.4 FP16 精度下 mlirGen 与 V100 平台前向卷积算子能效比 46
图 5.5 FP32 精度下 mlirGen 与官方算子库反向数据卷积性能对比 47
图 5.6 mlirGen 与 V100 平台反向数据卷积算子能效比 48

万方数据

表清单

表 3.1 Conv Dialect 计算组件 27
表 5.1 硬件参数对比 41
表 5.2 输入参数形式 42

万方数据

1 绪论

1.1 研究背景及意义

近年来卷积神经网络（CNN）[1]在深度学习中具有重大作用，特别是在计算机视觉领域[2]。它通过多层次的卷积层、池化层等结构，能够自动从数据中提取具有代表性的特征。这一特性使得 CNN 在图像分类[3]、目标检测[4]、语义分割[5]、视频分析[6]等任务中表现出了出色的效果。CNN 利用局部感受和共享权重的机制，不仅减少了计算和内存开销，还能识别图像中的不同位置和尺度的特征。此外，CNN 在各类任务中展现出强大的自动特征学习能力和空间不变性，使得它能够应对复杂的图像和视频内容。随着大规模数据和硬件加速的发展，CNN 的应用领域也扩展到了自然语言处理[7]、医学影像分析[8]等多个领域，成为现代人工智能技术的重要基础。

卷积运算[9]作为卷积神经网络的核心计算单元，直接影响整个网络的计算效率和性能表现。由于卷积操作涉及大量的矩阵乘法与加法，其优化对于提升深度学习模型的训练速度和推理效率至关重要。高效的卷积实现不仅能够加速模型训练，缩短训练时间，还能在推理阶段有效降低计算延迟，以满足实际应用对实时性和吞吐量的需求。为此，近年来各类深度学习框架（如PyTorch[10]、TensorFlow[11]等）纷纷针对特定硬件架构进行人工深度优化，以减少卷积计算的耗时来提升整体性能。这些优化方法通常依赖于底层硬件特性，例如缓冲大小、内存带宽等。然而，随着硬件平台的快速迭代与深度学习框架的不断演进，手写算子往往需要持续地维护与更新，以适应新的计算需求和硬件特性。这导致开发成本和维护难度显著增加，尤其是在需要支持多种硬件平台的情况下，手写算子的优化复杂度呈指数级增长，进一步加重了维护负担。

在这个背景下，MLIR[12]作为一种新型的中间表示语言，为卷积算子的生成和优化提供了新的思路。MLIR 具备多层次、可扩展的特点，使其能够支持多个硬件平台的优化与代码生成。相较于传统依赖手写底层代码优化的方式，MLIR 提供了一种硬件无关的中间表示，使开发者可以借助 MLIR 框架高效地优化和生成卷积算子，从而适配不同的计算硬件。MLIR 的设计理念使得卷积算子的实现不再依赖于针对每种硬件平台进行手动调优，而是通过编译器的自动化优化

万方数据

Pass 生成高效的硬件代码，极大地降低了开发的复杂性，提高了代码的可维护性和可复用性[13]。

MLIR 的核心优势在于其对多种硬件平台的支持能力。它能够将卷积算子转换为与具体硬件无关的中间表示，并通过多层次优化 Pass 自动生成针对特定平台的优化代码。这样一来，开发者无需深入了解每种硬件架构的细节，即可借助 MLIR 生成高效、适配不同硬件的计算代码。此外，MLIR 具备良好的可扩展性和灵活性，允许开发者针对特定硬件平台扩展新的优化策略，以更好地适配其特性。MLIR 不仅支持 GPU[14]、CPU[15]等通用计算硬件，还能够针对 FPGA[16]、ASIC[17]等定制化硬件进行优化，从而显著提升卷积算子的移植性与通用性。

尽管 MLIR 在卷积算子的优化与生成方面展现出显著优势，其在实际应用中的性能与潜力仍需在特定硬件平台上进一步验证。近年来，国产加速器作为 AI 计算的关键硬件，正逐步成为推动国内 AI 技术发展的重要支撑力量[18]。较于主流 GPU、TPU[19]等加速器，国产加速器采用独特的硬件架构和指令集，在计算模式、内存管理以及数据传输等方面存在差异。

因此，如何将 MLIR 的优势与国产加速器的特性相结合，自动生成适配国产加速器的高效卷积算子的研究具有重要的理论价值和实践意义。它不仅解决了传统手写卷积算子在跨平台移植和硬件适配方面的难题，还能够为国产加速器的算子优化提供新的解决方案。通过这一研究，不仅能够提高卷积算子的执行效率，还能够推动国产硬件在 AI 领域的广泛应用，为国内 AI 产业的自主创新和技术突破提供有力支撑。

1.2 国内外研究现状

1.2.1 MLIR 研究现状

MLIR 作为一种新型的编译基础设施，近年来在学术界和工业界均受到了广泛关注，其核心优势在于高度的可扩展性和模块化设计，为构建领域特定编译器提供了有力支持[20]。MLIR 通过定义丰富的中间表示层次和可插拔的转换机制，使得开发者可以针对不同的计算领域设计专用的 Dialect，从而在高层抽象与低层硬件优化之间架起一座桥梁。以下是近年来基于 MLIR 的若干代表性研究工作的详细扩展与探讨。

2020 年，U Bondhugula[21]等人首次尝试基于 MLIR 构建一个矩阵乘法的端

万方数据

到端编译器。在这项工作中，研究者充分利用了 MLIR 中预先定义的公共组件，构建了一套描述矩阵乘法逻辑的中间表示。在编译过程中，编译器不仅仅完成了基本的语义降级，还集成了矩阵分块、循环展开、向量化等一系列优化算法，从而极大地提升了计算效率。此项工作标志着 MLIR 在高性能计算领域的应用潜力，证明了其能够通过灵活的 IR 转换机制实现复杂算法的高效编译与优化。

在 2021 年，Vivek Khandelwal[22]等人利用 MLIR 的抽象机制构建了一个实验性的降级流水线，专门针对 NVIDIA GPU 的张量核心进行优化。该流水线能够自动生成矩阵-矩阵乘法的代码，直接映射到 GPU 硬件的张量运算单元。实验结果显示，在 Geforce 3090 RTX 上，生成的代码在不同测试场景下分别达到了 CuBLAS[23]性能的 95%至 119%以及 80%至 160%。这一成果不仅展示了 MLIR 在跨平台硬件优化中的灵活性，也表明通过抽象和自动降级，可以有效缩短开发周期，并在性能上与传统高度优化库形成竞争。

2022 年，Henrik Barthels[24]等人提出了一种名为 linnea Dialect 的新型中间表示，用于专门表达线性代数计算。该 Dialect 不仅包含了对矩阵属性的详细描述，还设计了一整套高级算法转换规则。通过将 linnea Dialect 无缝集成到 MLIR 框架中，整个编译过程可以实现端到端的线性代数计算降级：从高层的数学表达逐步转换到框架中已经提供的低级别方言。

2023 年，Tiago Trevisan Jost[25]等人针对医学仿真领域的心脏电生理模型，基于 MLIR 构建了一个端到端的 GPU 代码生成编译器。在该项目中，编译器首先对心脏电生理仿真中的复杂计算任务进行了抽象描述，随后通过 MLIR 的多级降级机制，自动生成适用于 GPU 的高效并行代码。实验数据表明，在 Nvidia A100 GPU 与 36 核 AVX-512 Intel CPU 的对比测试中，生成的 GPU 代码在执行速度上比经过矢量化优化的 CPU 代码快 3.17 倍，同时在每瓦浮点运算性能上超过 CPU 超过 7 倍。与此同时，Abhinav Kumar[26]等人也在 2023 年基于 MLIR 开发了一款面向数字信号处理(DSP)的特定编译器。该编译器通过定义全新的 DSP Dialect，能够精确描述数字信号处理领域中的各类算法和数据流特性。借助于 MLIR 提供的灵活优化框架，该 DSP 编译器不仅实现了针对性强的代码优化，还在实例算法上展示了高达 10 倍的性能提升。

总体而言，从 2019 年 MLIR 的首次亮相到现如今的广泛应用，示例证明了 MLIR 在领域特定编译器中的潜力，其模块化和可插拔的架构设计，使得开发者能够快速构建和集成各类领域专用的编译器前端、优化中间层以及后端代码生

万方数据

成器。通过这种方式，不仅能够实现高效的算法转换，还能根据不同硬件平台的特性进行定制化的性能优化。

1.2.2 国内外加速器现状

近年来，随着人工智能、大数据[27]和云计算[28]等技术的迅猛发展，GPU 作为高性能计算[29]和深度学习[30]领域的核心硬件，其重要性日益凸显。在全球范围内，NVIDIA 和 AMD 是两家知名的 GPU 制造商，各自推出了多款具有代表性的 GPU 产品，并在计算领域尤其是人工智能和高性能计算中占据了重要位置。NVIDIA 的代表性 GPU 产品包括基于 Volta 架构[31]的 V100 系列、基于 Ampere 架构[32]的 A100 系列以及最新的基于 Hopper 架构[33]的 H100 系列等。这些 GPU 凭借强大的算力性能和 NVIDIA 独有的 CUDA 技术[34]，在深度学习、数据分析、科学计算等领域广泛应用。特别是 CUDA 技术，它为开发者提供了强大的并行计算能力，使得 NVIDIA 的 GPU 在人工智能训练和推理过程中具有显著的优势。另一方面，AMD 则通过其 RX 系列 GPU[35]，如 RX 9070、RX 6800 XT 等。与 NVIDIA 的 CUDA 技术相比，AMD 采用了 ROCm 平台[36]，这是一个开源的异构计算平台，支持多种编程语言和框架，旨在为开发者提供一个高效的计算环境。ROCm 平台的开源性质，使得开发者能够在 AMD 的 GPU 上灵活地进行计算优化，尤其是在机器学习和科学计算等高性能计算任务中，逐渐展现出强大的竞争力。

国内 GPU 产业起步较晚，但近年来发展迅速。自 2014 年 4 月景嘉微成功研发出我国首款高性能、低功耗 GPU 芯片 JM5400[37]以来，国产 GPU 行业正式迈入新的发展阶段。目前，国内 GPU 产业链已初具规模，主要涵盖以下几类国产 GPU 加速器：寒武纪系列[38]、摩尔线程系列[39]、华为异腾系列[40]等。这些国产 GPU 加速器已在高性能计算、深度学习、大数据处理等领域得到广泛应用，为我国 GPU 产业的发展提供了有力支撑。

万方数据

1.3 研究内容

根据上述讨论，卷积运算作为卷积神经网络的核心计算单元，直接影响网络的计算效率。然而，手动优化各种不同参数的卷积算子成本高昂，不仅后续优化与维护需要大量人力投入，还面临跨平台移植与硬件适配的挑战。因此，自动化生成卷积算子显得尤为重要。本文聚焦于 MLIR 的优势与国产加速器的特性，设计并实现了一种适配国产加速器的 MLIR 卷积算子生成器，旨在降低卷积算子的优化成本，同时为国产加速器的算子优化提供创新性解决方案。本文的主要研究内容如下：

(1) 基于 MLIR 构建了一套分层卷积算子生成框架，充分发挥了 MLIR 在灵活性和多层次表示上的优势。在框架设计中，采用了分层的下降通路，实现了不同粒度的优化。该框架支持在高层进行算法建模，并通过层次化转换，将高层计算表达逐步降解为底层硬件可识别并高效执行的指令序列，从而充分发挥硬件的并行计算能力，实现高效计算优化。

(2) 基于 Implicit GEMM 算法，设计了 Conv Dialect，以实现卷积在类 GPU 上的计算逻辑。在设计过程中，Conv Dialect 充分考虑了卷积计算的复杂性和层次结构，通过引入大量的 Operation 设计，能够分层次地描述卷积计算任务。这种分层式设计有助于在不同计算阶段进行优化，同时也便于将高层计算模型逐步映射为低层硬件执行指令。Conv Dialect 特别注重类 GPU 架构的特性，通过分装底层指令来针对性地进行优化。

(3) 采用矩阵外积算法、双缓冲流水并行等优化技术，并结合 MMAC 等底层指令，构建高效的优化通路，以提升卷积运算的执行效率。首先，矩阵外积算法通过修改矩阵乘法维度顺序，降低了内存的访问次数，提高的整体性能。其次，为了降低传统“先取数后计算”模式带来的访存延迟，采用了双缓冲技术。该技术充分利用国产加速器访存单元与计算单元并行执行的特性，通过设置两个缓冲区，实现访存与计算操作的交替进行。这样，计算过程中能够同时预取数据，有效地隐藏了部分内存访问延迟，从而提升整体运算效率。最后针对 FMA 指令的处理大矩阵时的频繁使用导致的计算瓶颈问题采用 MMAC 指令优化底层矩阵乘法，进一步提升计算性能。

万方数据

1.4 论文组织架构

本文主要探讨如何结合 MLIR 的优势与国产加速器的硬件特点，设计并实现一个多层次卷积算子生成器，从而实现卷积操作的自动化生成。通过分析卷积操作的特性，并充分考虑国产加速器的硬件架构，本文设计了生成器的整体架构、下降通路与优化通路，以确保生成器能够在国产加速器上自动生成高性能的卷积算子。本文将按以下结构进行详细阐述：

第一章作为本文的绪论部分。首先介绍了本文的研究背景和意义，然后详细描述了当下国内外加速器的研究现状和 MLIR 的研究现状。最后对本文的研究内容和结构进行了概述，明确了每一章的重点和目的。

第二章介绍所依赖的理论基础和技术框架。首先介绍了 MLIR 的技术理论，包括其核心概念、Dialect（方言）、Pass 和 Operation（操作）的设计和实现。接着，本章介绍了卷积加速算法，重点分析了两种常用的加速算法：im2col + GEMM 和 Implicit GEMM 算法。通过详细分析这两种算法的计算过程及其优缺点，最终选择了 Implicit GEMM 算法作为卷积算子生成器的理论基础，因其在硬件加速平台上具有更好的适应性和性能表现。最后，本章还介绍了国产类 GPU 加速器的架构设计、存储模型和线程管理模型。

第三章主要阐述了 MLIR 卷积算子生成器（mlirGen）的框架设计。首先，从设计理念和分层架构的角度对 mlirGen 进行了整体描述，明确了各个模块之间的关系。然后，重点介绍了 Conv Dialect 的设计，这一 Dialect 是专门为卷积操作量身定制的，它能够有效地表达卷积算子的计算逻辑，并与下层硬件架构相结合。接着，本章详细讨论了生成器的下降通路，重点介绍了 Conv Dialect 内部的 Operation 下降和 Conv Dialect 的下降。

第四章介绍了卷积算子生成器所采用的优化算法与底层优化指令，主要包括矩阵外积算法、双缓冲流水并行算法、MMAC 指令。首先，对当前存在的问题进行了深入分析，然后结合国产加速器的硬件特性，通过描述具体的算法设计与指令执行流程，探讨其优化效果与实现方式。

第五章对 mlirGen 的性能进行测试。首先，介绍了实验的测试环境和测试用例的设计，以确保测试结果的可靠性。接着，定义了多平台性能对比的评测标准。随后，在国产加速器和 NVIDIA V100 上分别进行卷积前向计算和反向数据卷积

万方数据

的性能测试，并对实验结果进行了详细分析。

第六章对本文的工作进行了回顾和总结，最后对未来的研究方向和内容进行了展望。

万方数据

2 相关理论与技术基础

本章主要阐述了与本文设计相关的理论与技术基础。首先介绍了 MLIR 的基本概念，并对其 Dialect、Pass 和 Operation 等进行了详细讲解；随后，深入探讨了卷积加速算法的研究进展，分析了不同优化策略和实现方法；最后，重点分析了国产加速器的架构，并深入探讨了加速器的存储模型和线程结构。

2.1 MLIR 技术理论基础

2.1.1 MLIR 概述

MLIR 是一个由 Google 提出的开源编译框架，旨在为各种编译器优化和硬件平台的代码生成提供一个统一、灵活、可扩展的中间表示（Intermediate Representation, IR）[41]。它的核心目标是通过支持多层次的中间表示，解决编译器在面向机器学习和高性能计算时面临的复杂性问题，并在不同硬件平台之间提供更高效的优化。随着 AI 计算和硬件加速的需求日益增长，传统的编译框架（如 LLVM[42]）虽然能够处理通用计算任务，但在处理机器学习、深度学习等应用时却面临诸多挑战，尤其是涉及到多个硬件平台（CPU、GPU、TPU、FPGA 等）的优化时，现有的框架难以提供统一而高效的支持。MLIR 应运而生，它采用了多层次的 IR 架构，能够在不同抽象层次之间进行优化，并支持多种硬件架构的编译任务，从而为机器学习和高性能计算提供强有力的支持。

MLIR 的设计理念非常注重可扩展性和灵活性。与传统的 IR 不同，MLIR 并不是单一的表示，而是通过引入多种不同的“方言”（Dialect）来实现对不同领域的支持。例如，MLIR 可以为 TensorFlow 提供一种专门的方言，用于表示 TensorFlow 的计算图，支持高效的图优化和硬件映射。同时，MLIR 还可以为其他计算框架、应用领域甚至新的硬件架构提供定制化的方言，确保它们在相同的编译框架下运行和优化。每种方言都可以拥有自己的操作（Operation）和类型系统，这些操作符代表了计算图中的基本计算单元，通过这些操作符，MLIR 能够表示不同领域的计算任务，并提供相应的优化和转换机制。图 2.1[43]展示 MLIR 的 Dialect 转换图。

万方数据

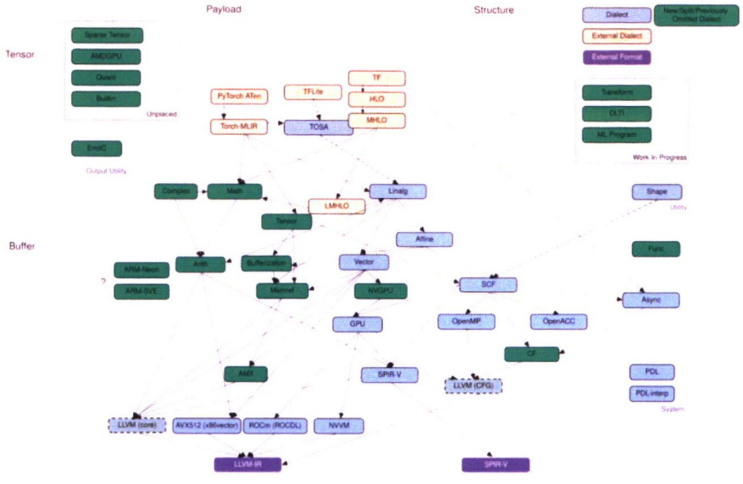


图 2.1 MLIR 的 Dialect 转换

2.1.2 MLIR Dialect 介绍

在 MLIR 中，Dialect（方言）作为一个核心概念，起到了定义和扩展中间表示（IR）的重要作用，旨在满足不同领域计算需求。每个 Dialect 都根据其特定应用场景，通过一系列组件来抽象领域中的计算任务。如图 2.2 所示 Dialect 包括操作（Operation）、类型系统（Type）、属性（Attributes）以及控制流结构（Region 和 Block）。

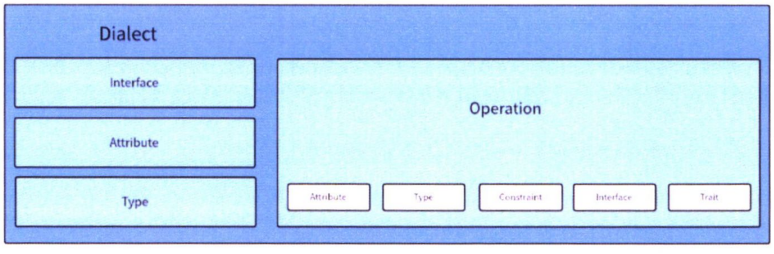


图 2.2 Dialect 结构

Operation 是 Dialect 的基础构件，表示计算任务中的基本操作。每个 Operation 有一组输入和输出，定义了操作数的类型以及操作本身的功能。例如，TensorFlow Dialect 中可能定义了表示张量乘法、加法等常见操作的 Operation，而针对 GPU

万方数据

优化的Dialect则可能定义特定的矩阵乘法、向量计算等Operation。每个Operation不仅仅是一个单纯的计算操作，它还具有丰富的元数据（如Attributes、Region、Traits等），这些元数据有助于在编译时执行优化。Operation通常与特定的类型（Type）结合使用，确保输入输出数据的兼容性。

Type是Dialect内部非常重要的一部分，它定义了Operation操作数的结构和数据格式。不同的Dialect会根据其应用领域设计不同的类型系统，例如，在深度学习框架中，TensorFlow方言的类型系统可能会包含张量类型（Tensor），而在图像处理领域，可能会有图像数据类型。Type确保了Operation在计算过程中使用的数据符合预期，并为后续的优化提供了必要的结构。在MLIR中，Type的设计非常灵活，可以支持标量、向量、矩阵等多种数据类型，还能支持用户自定义类型，满足不同领域对数据的特殊需求。

Attributes是附加到Operation上的额外信息，它们提供了关于操作的额外上下文或常量值。例如，在卷积操作中，可能会有与卷积核尺寸、步长、填充方式相关的属性。Attributes可以是标量值、字符串或更复杂的数据结构，它们在编译优化过程中起到了重要的作用。优化器可以利用这些属性来调整计算过程，从而提高性能，比如在卷积操作中优化内存布局，或在矩阵运算中进行操作符融合。

Region和Block是用于表示复杂控制流的结构。在MLIR中，Operation不仅仅代表一个简单的计算操作，它们还可以包含控制流信息。每个Operation可以有多个Region，每个Region内部包含一个或多个Block。Block是代码的基本执行单元，类似于传统编译器中的基本块，它是一个包含一组指令或操作的容器。Region和Block使得MLIR能够表示更复杂的计算结构，如循环、条件分支等。对于涉及复杂控制流的计算任务（如图算法、循环优化等），Region和Block提供了灵活的表达能力。

通过这些组件，Dialect不仅能够描述特定领域的计算任务，还能够通过定义特定的操作符、类型、属性以及控制流结构，为编译器优化提供丰富的上下文。MLIR中的每个Dialect都可以根据具体需求定制和扩展，支持从高层计算到硬件级别的优化，最终帮助生成高效的目标代码。Dialect的设计让MLIR不仅能够支持传统的CPU和GPU计算，还能够适应新的硬件架构，如TPU、FPGA等，从而为多样化的计算任务提供了强大的支持。

万方数据

2.1.3 MLIR Pass 概述

MLIR Pass（通路）是 MLIR 中执行变换和优化的基本单元，它为编译器优化提供了强大的灵活性和可扩展性。Pass 是对中间表示（IR）的处理过程，通过 Pass 实现 Operation 之间的转换。每个 Pass 都会对输入的 Operation 执行一系列转换，最终输出经过优化或改进的 Operation。这一过程可以是简单的操作符替换，也可以是复杂的控制流优化或数据流分析。为了使 MLIR 能够更好地扩展和适应复杂的编译需求，它引入了两个重要的抽象概念：Trait（特性）和 Interface（接口）。这两个概念的引入解决了 MLIR 在实际应用中面临的可扩展性和灵活性问题。

在 MLIR 的编译架构中，Trait 主要用于限定某个 Pass 作用的 Operation 子集。简单来说，Pass 并不是对所有 Operation 都进行优化，而是通过对 Operation 的 Traits 进行筛选，判断该 Operation 是否满足条件，进而决定是否应用目标 Pass。例如，如果一个 Operation 具有某种 Trait，Pass 就可以根据这些 Trait 来选择性地应用优化策略。另外，Interface 则为多个 Operation 或一个 Region 的应用提供了更高层次的灵活性。Interface 的作用是限制和定义 Pass 在特定 Region 中的应用范围，确保 Pass 只会在满足特定条件的 Region 内被触发。这意味着，编译器可以根据 Region 的上下文信息来判断是否可以应用某个 Pass，从而提高编译效率，并避免不必要的计算和资源消耗。通过 Trait 和 Interface 的结合使用，MLIR 能够更加灵活地管理和调度优化过程，使得编译器能够在不同的硬件架构和应用场景下，执行高效且精确的优化。

总的来说，MLIR 中的 Pass 机制为编译器提供了一种强大的方式来执行复杂的优化和变换。它不仅通过 Trait 和 Interface 增强了可扩展性，还通过精确的 Operation 选择和区域限制，保证了优化过程的高效性和灵活性。这些特性使得 MLIR 能够处理多种类型的中间表示，并在编译过程中不断改进和优化生成的代码，最终实现更好的性能和资源利用率。

2.1.4 MLIR Operation 概述

在 MLIR 中，Operation 是编译过程中的核心构成单元，它代表着一个具体的计算步骤或数据变换。每个 Operation 通过几个关键部分来定义其行为、数据流动和计算规则，如图 2.3 所示主要包括 Name、OpType、Operands、Results 和

万方数据

Attributes。这些部分协同工作，共同确保了 Operation 能够在编译过程中执行预定的计算任务，同时为优化和代码生成提供了灵活性和可扩展性。

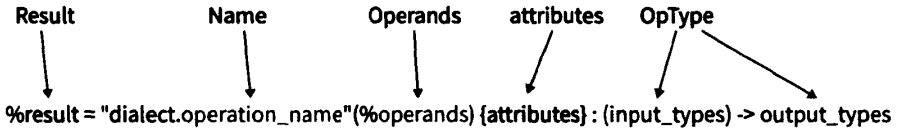


图 2.3 Operation 格式

在 MLIR 中，Name 可以视为 Operation 的“标识”或“类别”，用于定义 Operation 执行的计算逻辑。例如，Name 可以表示加法（Add）、乘法（Mul）、矩阵乘法（MatMul）等算子。编译器通过解析 Name，能够确定适用的优化策略，如循环展开、内存访问优化和常量折叠等。Operands 和 Results 分别表示 Operation 的输入和输出数据，而 OpType 则定义了输入输出数据的类型。在 MLIR 中，操作数不仅限于基本数据类型，还可扩展至 Tensor 等复杂数据结构。Attributes 用于存储 Operation 的静态元信息（如卷积核尺寸、步长等），这些属性在运行时不可变，但为编译器优化和代码生成提供了关键上下文。

通过这些元素，MLIR 能够高效地表达各种计算任务，从简单的算术操作到复杂的矩阵运算，并为不同硬件平台提供定制化的优化支持。MLIR 的模块化和可扩展性设计，支持在多种硬件架构上高效执行，同时简化了编译器开发，减少了针对特定平台的重复工作，并提升了计算任务的执行效率。

2.2 卷积加速算法

2.2.1 im2col + GEMM 算法

im2col + GEMM 算法[44]是一种通过将卷积操作转化为矩阵乘法来加速卷积计算的优化方法，其核心思想是利用矩阵乘法的高效计算方式替代传统的卷积运算，从而显著提升卷积操作的性能。传统的卷积操作需要通过滑动卷积核在输入图像上进行逐元素的乘加运算，这种计算方式不仅在处理时依赖于复杂的局部区域计算，而且在多次数据访问中可能会带来较大的内存访问开销。此外，传统卷积还面临着计算过程中大量重复的数据访问和局部存储的效率低下问题，特别是在多核或 GPU 等并行计算架构下，这些因素会导致处理性能的瓶颈。

万方数据

im2col + GEMM 的关键创新在于首先通过滑动卷积核遍历输入图像，如图 2.4 所示将每个局部区域展开成一列向量。这些局部区域的列向量将被拼接成一个大的矩阵，称为“im2col 矩阵”，这个矩阵的每一列都代表一个感受野（即一个局部卷积区域），而列向量的数量则等于输出特征图中的位置数。接下来，将卷积核的权重矩阵展开成一个二维矩阵，其中每一行代表一个卷积核的所有权重。随后，通过对这两个矩阵执行矩阵乘法，即可完成卷积计算，从而避免了传统卷积的局部计算开销。这种将卷积运算转化为矩阵乘法的操作极大地优化了计算性能，因为矩阵乘法本身是高度优化的，许多现代计算库为其提供了针对不同硬件平台的优化实现，能够在 GPU 和 CPU 上发挥出色的性能。此外，矩阵乘法的内存访问模式通常是连续的，这与卷积操作中的局部数据访问模式不同，使得矩阵乘法能够更有效地利用高速缓存，减少内存访问延迟，提高数据吞吐量，从而充分发挥硬件计算能力。

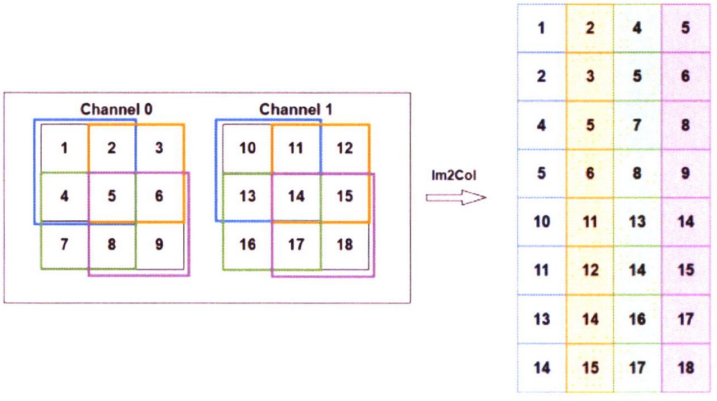


图 2.4 im2col 算法

im2col + GEMM 不仅提高了计算效率，而且减少了内存占用，优化了数据访问模式，使得卷积操作在大规模并行计算环境下能够得到高效处理。在实际应用中，该方法被广泛用于计算机视觉、自然语言处理和语音识别等领域的深度学习模型，尤其是在涉及大规模卷积运算的卷积神经网络（CNN）中。此外，许多现代深度学习框架，如 TensorFlow 和 PyTorch，都已将 im2col + GEMM 作为卷积操作的标准实现，充分利用了其对现代硬件平台的优化，使得训练和推理过程更为高效。

万方数据

2.2.2 Implicit GEMM 算法

Implicit GEMM（隐式卷积）[45]是一种用于加速卷积神经网络中卷积计算的高效方法，其核心思想是通过将卷积操作映射为等效的矩阵乘法，在不显式展开输入特征图的情况下，直接在计算过程中完成卷积运算，从而显著提高计算效率。传统的 im2col 方法通过将输入特征图重排为一个较大的矩阵，使得每个局部区域的卷积计算可以转化为矩阵乘法问题，从而能够利用优化良好的矩阵乘法库进行加速。然而，这种方法需要显式展开输入数据，导致了额外的内存消耗和数据搬运成本，尤其是在处理大规模数据时，这种内存重排操作会显得尤为昂贵。更重要的是，im2col 方法依赖于内存的线性布局，而这种布局可能会对硬件的缓存效率[46]和内存带宽[47]造成不利影响，从而影响整体的计算性能。

相比之下，Implicit GEMM 则通过巧妙的索引计算，在计算过程中直接访问输入数据，并在一次迭代中完成卷积操作。这意味着隐式卷积不需要显式的内存重排，能够直接在原始数据上进行计算，避免了不必要的内存占用和数据搬运。具体来说，隐式卷积利用精心设计的数据访问模式和指针更新策略，通过在共享内存中动态构建卷积矩阵的分块，每个线程块负责加载输入数据和卷积核权重，并进行局部的矩阵乘法运算。这种方法不仅减少了内存占用，还优化了内存访问模式，使得计算过程中能更好地利用片上内存和硬件的带宽，从而提高计算效率。此外，隐式卷积特别适合于现代的并行计算架构，如 GPU 和 TPU。现代硬件加速器具有强大的并行计算能力和高速缓存机制，隐式卷积能够最大程度地利用这些特性。由于隐式卷积的计算过程不依赖于数据的显式重排，它能够更有效地使用硬件的并行计算资源，将卷积计算任务分配到多个计算单元中同时执行，从而实现更高的吞吐量。

总体而言，Implicit GEMM 算法通过巧妙的内存访问模式和并行计算策略，最大限度地利用了现代硬件的计算和内存资源，避免了显式展开输入数据带来的内存消耗和数据搬运问题，从而大幅提升了卷积计算的效率。与传统的 im2col 技术相比，隐式卷积不仅减少了内存占用和数据传输成本，还能更好地匹配硬件架构的特性，发挥并行计算资源的最大效能，尤其在大规模数据并行计算和深度学习任务中，隐式卷积展现出了巨大的优势。因此文章使用 Implicit GEMM 算法作为卷积算子生成器的理论基础。

万方数据

算法 2.1 二维 Implicit GEMM 算法

```
Input: 输入矩阵 x[n][c][h][w]，卷积核 w[k][c][R][S]
Output: 输出矩阵 y[n][k][outh][outw]
  M= k  // 输出通道数
  N = n * outd * outh * outw
  K =c * R * S
  for i= 1 to M:
      k' = i
      for j = 1 to N:
          n' = j / (*outh*outw)    //输入批次索引
          j_res=j  %  (outd*outh*outw)
          oh=(j_res % (outh * outw)) / outw
          ow=(j_res % (outh * outw)) % outw
          for m = 1 to K:
            c' = m / (R * S)  // 输入通道的索引
            m_res = m % (R * S)
            fh = m_res % (R * S) / S   // 计算输出的高度偏移量
            fw = m_res % (R * S) % S   // 计算卷积核的宽度偏移量
            ih = oh *strid_h – pad_h +fh
            iw = ow *strid_w – pad_h +fw
            accumulator = accumulator + x(n,c',ih,iw) * w(k,c',fh,fw)
          end for
      end for
  end for
  y(n',k,oh,ow) = accumulator
```

2.3 国产加速器

2.3.1 加速器架构

本文所使用的加速器是一种类 GPU 加速器，它通过高速的 PCI-E 总线[48]与 CPU 紧密相连，实现了数据的高效传输和高速交互。与传统 GPU 相比，类 GPU 加速器在架构设计上进行了针对性的调整，以更好地适应高性能计算和人工智能等领域的需求。如图 2.5 所示，类 GPU 加速器包含多个关键组件：计算单元阵列（Compute Unit，简称 CU）[49]、分级缓存（L1 和 L2 Cache）、全局内存

万方数据

（Global Memory）[50]以及 SIMD（Single Instruction Multiple Data）计算单元[51]。这些组件相互配合，共同提升计算性能和数据处理能力。CU 是加速器的核心计算模块，每个加速器通常集成多个 CU，以实现大规模并行计算能力。每个 CU 内部进一步包含 4 个 SIMD 单元，每个 SIMD 单元能够并行执行多个线程束（Wavefront）[52]，通常最多支持 10 个线程束并行运行，每个线程束内部包含 64 个线程。SIMD 的核心计算组件是矢量计算单元（vALU）[53]，这些 vALU 能够同时进行整数运算和浮点运算，从而支持复杂的科学计算、图形渲染以及深度学习推理等多种任务。通过 SIMD 架构，加速器能够在单个指令周期内对多个数据进行并行处理，大幅提高计算吞吐量。

为了提升计算任务的调度效率，每个 CU 还配备了标量计算单元（sALU）和专用寄存器（SGPR）。sALU 主要用于执行标量运算，例如地址计算、分支判断以及指令调度等任务。相较于 vALU 并行处理多个数据，sALU 主要执行独立的控制逻辑，确保计算任务的顺序性和数据流的合理调度。为了优化资源利用率，一个 sALU 单元通常可以被同一 CU 内的四个 SIMD 单元共享，这种共享机制有效减少了额外的硬件开销，同时提升了计算单元的执行效率。此外，CU 内的专用寄存器（SGPR）用于存储线程执行所需的关键数据，并在多个计算单元之间提供快速访问，从而降低数据存取延迟，提高计算效率。整体而言，类 GPU 加速器采用了高度并行的计算架构，并结合高效的数据管理机制，使其在高性能计算领域具有优势。

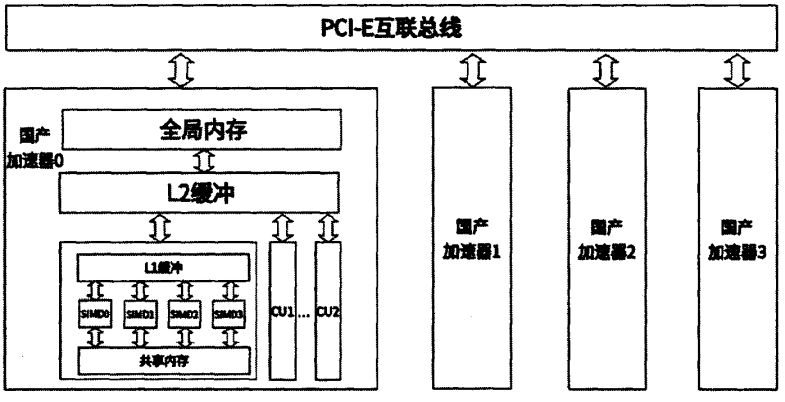


图 2.5 国产类 GPU 加速器架构

万方数据

2.3.2 存储模型

在类 GPU 加速器的内存层次结构中，最常用到的可供编程的存储器包括全局内存(Global Memory)、共享内存(Shared Memory)[54]和寄存器[55]。这三类存储器在访问速度与存储容量上各具特点，合理规划和利用不同层次的内存对于优化类 GPU 加速器程序的性能至关重要。

全局内存是类 GPU 设备上最基础的存储资源之一，所有线程块（Block）[56]内的线程均可访问该内存区域。作为类 GPU 中容量最大的存储类型，全局内存能够提供大规模数据存储，适用于存放全局共享的数据及生命周期较长的变量。然而，由于全局内存通常位于类 GPU 的显存中，其带宽远低于寄存器和共享内存，并且访问延迟较高，因此往往成为影响线程性能的主要瓶颈。为提高全局内存的访问效率，可以利用合并内存访问技术，将多个线程对连续地址的访问整合为单个内存事务，以减少访问次数，降低总的数据传输开销。例如，在一个线程块中，如果相邻的线程能够按照特定模式访问连续的内存地址，GPU 可以将这些访问合并成一次事务，而非多个独立的请求，从而减少内存带宽的占用。此外，还可以通过优化内存对齐策略、延迟隐藏技术等策略，降低内存访问冲突与延迟，从而提升程序整体性能。

共享内存又称 LDS（Local Data Share）是线程块内部的一块高速缓存区域，供同一线程块内的所有线程共享访问。与全局内存不同，共享内存位于计算单元内部，因此访问速度极快。由于其容量有限，仅有 64KB 的片上内存，因此共享内存主要用于存储局部数据，特别是需要多个线程频繁访问的临时计算结果或数据缓存。当多个线程需要频繁访问相同的数据时，合理使用共享内存可以有效减少对全局内存的访问压力，从而大大提高程序的执行效率，尤其在矩阵运算、图像处理等高并发场景中尤为重要。尽管共享内存具备高性能优势，但使用时需要注意 Bank 冲突[57]问题。共享内存被划分为多个 Bank，每个 Bank 允许并行处理一个读取或写入操作。然而，当线程束内的多个线程访问同一 Bank 的不同地址时，会导致访问串行化，进而降低性能。为减少 Bank 冲突，可以采用数据重排或内存对齐等优化策略。数据重排通过调整数据存储布局，使线程访问不同的 Bank，从而减少冲突；内存对齐则确保数据按特定边界存放，避免多个线程同时访问同一 Bank 的不同位置。通过这些优化措施，可以充分发挥共享内存的性能优势，避免 Bank 冲突带来的性能下降，进而提升程序整体执行效率。

万方数据

寄存器是 GPU 中用于存储临时数据和计算结果的高速存储单元，位于每个线程的执行单元内部，并与计算核心直接连接。由于寄存器的访问不需要通过片上缓存或外部存储，其读写速度极快，远优于共享内存、L2 缓存或全局内存。因此，寄存器在 GPU 计算中扮演着至关重要的角色，主要用于存放线程执行过程中频繁访问的数据，如中间计算结果、常量、循环变量以及其他临时变量。然而，每个线程可使用的寄存器数量是有限的，在国产类 GPU 加速器中每个线程最多可以使用 256 个 VGPR 寄存器来存储私有数据。如果某个线程对寄存器的需求超过了硬件分配的上限，编译器可能会将部分数据溢出至全局内存，从而引入较高的访问延迟，影响计算性能。因此，在优化程序时，需要合理管理寄存器资源，以在计算性能和并行度之间取得平衡，确保线程能够高效执行，同时避免寄存器溢出带来的性能下降[58]。

L1 Cache 和 L2 Cache 是两种重要的缓存[59]层次，用于提高内存访问效率和加速计算。L1 Cache 是计算单元内部的高速缓存，紧密集成于每个计算单元中，主要用于存储来自全局内存或共享内存的热点数据。由于其访问速度极快，L1 Cache 能有效降低全局内存访问延迟，特别是在多个线程访问相同数据时，可以减少重复的数据读取，从而提升计算效率。L2 Cache 则是整个 GPU 范围内的共享缓存，位于多个计算单元之间，容量比 L1 Cache 更大。它可以缓存来自不同线程块和计算单元的数据，从而减少对全局内存的直接访问压力，并在多个计算单元之间提供高带宽数据共享。L2 Cache 的存在使 GPU 在处理大规模数据时能够保持高效的数据访问，尤其在数据访问模式较为分散的情况下，能够显著提升整体计算性能。

2.3.3 线程模型

为了帮助开发者更高效地组织和管理线程，国产加速器在并行计算架构上借鉴了 CUDA 的三层线程层次结构，如图 2.6 所示。该结构将线程的组织方式抽象为线程网格（grid）、线程块（block）和线程（thread）[60]，形成自上而下的层级关系。这种分层设计的核心理念是将大规模计算任务合理拆分为多个可并行执行的子任务，并结合硬件资源的特点进行高效调度，从而提升整体计算效率和资源利用率。

万方数据

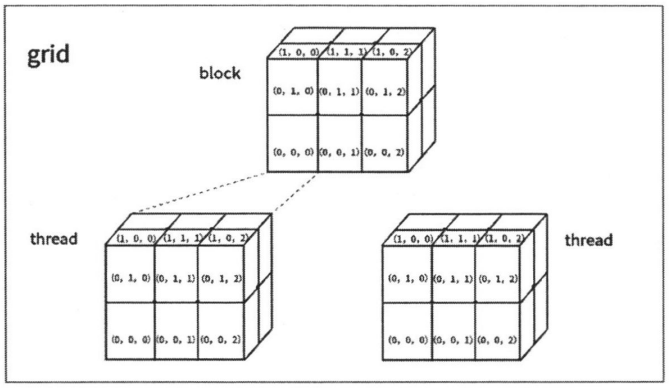


图 2.6 线程模型

线程是类 GPU 计算的最小执行单元，通常按数据并行的方式处理任务，每个线程负责处理数据的不同部分。每个线程有自己的寄存器，作为其私有存储空间，用来存放执行过程中的局部变量和中间结果；线程块由多个线程组成，是类 GPU 计算调度的基本单位。线程块内的线程共享 LDS 内的数据，并能够通过同步机制进行协作和通信。线程块中的线程通常按线程束进行调度和执行，每个线程束由 64 个连续线程组成，并遵循 SIMT（Single Instruction, Multiple Threads）[61]模式执行相同操作，即线程束内的所有线程在同一时钟周期内执行相同的指令。如果同一线程束内的线程执行不同的分支路径，则会引发线程束分化，导致部分线程处于闲置状态，降低计算效率；多个线程块集合在一起，构成线程网格，线程网格是类 GPU 计算任务的最高层级组织结构。整个网格中的线程块可以并行执行，且通常相互独立。由于类 GPU 的执行模型是基于线程块的调度机制，线程块的执行顺序通常是不确定的，每个线程块可以独立访问自己的私有寄存器和共享内存，并不会与其他线程块的私有内存产生冲突。

通过这种三层结构，类 GPU 能够在多个层次上进行并行计算，从而高效地利用硬件资源。开发者可以根据具体的任务需求和硬件配置，调整线程块和线程网格的大小以及布局方式，进一步优化计算性能，确保计算资源得到充分利用。

万方数据

2.4 本章小节

本章主要聚焦于实验所依赖的理论基础和技术框架。首先介绍了 MLIR 的基本概念，深入剖析了其核心组件，包括 Dialect、Pass 和 Operation 等，详细阐述了它们在编译优化过程中的关键作用。接下来讨论了卷积加速算法中的两种主流方法：im2col+GEMM 算法和 Implicit GEMM 算法。首先分析了 im2col+GEMM 的实现原理及其在卷积计算中的应用优势，接着对 Implicit GEMM 算法进行了深入探讨，解释了其在计算效率和内存利用方面的优化优势。经过比较和评估，最终选择 Implicit GEMM 算法作为本文卷积算子生成器的理论基础，以便实现更高效的加速效果。最后，本章详细介绍了实验所用国产类 GPU 加速器，包括其硬件架构设计、存储模型和线程管理模型。

万方数据

3 基于 MLIR 的卷积算子生成器的框架设计

本章将详细介绍 mlirGen，一个自顶向下的基于 MLIR 的卷积算子生成器，包括 mlirGen 的设计理念、整体架构、方言设计、下降通路等。同时，本章将对其中的关键技术，展开详细论述。

3.1 设计理念

mlirGen 的设计理念基于 MLIR 技术，其核心思想涵盖四个关键要素：正确性、灵活性、可扩展性和高效性，以适应现代硬件架构及不断演进的优化需求。首先，正确性是设计的首要原则。在卷积计算的编译与优化过程中，任何变换都必须确保计算结果的准确性，避免因优化引入误差，从而保证数值计算的可靠性；其次，灵活性是框架设计中的重要考量。卷积运算涉及多种参数配置，如维度、步长、填充方式等，因此框架需具备足够的适应性，以支持不同类型的卷积算法，满足多样化的计算需求；可扩展性则确保框架能够适应未来硬件与优化算法的发展。随着硬件架构的演进，新型硬件特性与底层指令集不断涌现，框架需要具备快速适配能力，充分利用新一代计算资源，实现高效计算调度；最后，高效性是框架设计的最终目标。mlirGen 生成的卷积算子必须高效利用硬件特性，以最大化计算资源利用率。无论是通过数据并行、指令调度，还是采用共享内存优化、流水线技术等策略，mlirGen 均致力于生成满足工业级性能需求的高效代码，帮助开发者在目标硬件上实现高效的卷积计算。

综上所述，mlirGen 旨在通过多层次的优化、灵活的架构设计以及对未来硬件发展的支持，构建一个高效且具有可持续扩展性的卷积算子生成平台，从而有效降低卷积算子在硬件适配和优化迭代过程中的成本。

3.2 整体架构

卷积算子的实现过程本质上是在硬件上根据算法逻辑，合理拼接并调度一系列底层指令，以实现高效的卷积计算过程。其核心难点在于如何在上层进行算法建模，同时又能够将这种上层的计算表达逐步映射为底层硬件能够识别与高效执行的指令序列，从而充分利用硬件的并行计算能力。为了实现这一目标，

万方数据

mlirGen 框架采用了一个分层设计的架构如图 3.1 所示，划分为四个关键阶段：算子生成层、建模层、实现层和执行层。在每一层之间保留足够的灵活性，同时又能保证最终的计算效率与硬件适配性。

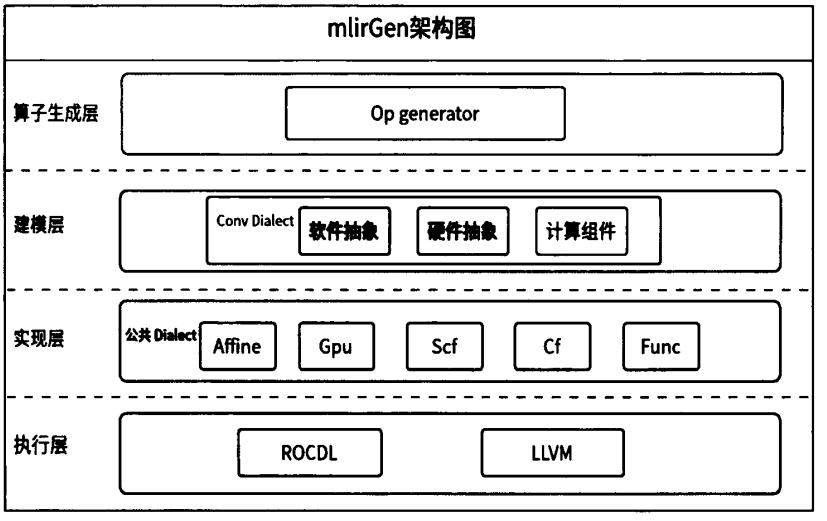


图 3.1 mlirGen 架构图

3.2.1 算子生成层

算子生成层是从使用者的角度出发进行设计的，旨在通过一组明确的参数输入，自动化地生成适用于特定硬件的高效卷积算子。以前向卷积为例，用户需要提供一系列关键信息，包括：输入特征图的参数——批次大小（N）、通道数（C）、高度（H）、宽度（W）；卷积核的参数——数量（K）、高度（R）、宽度（S）；卷积超参数——步长（U,V）、填充（P,Q）；数据类型（如 FP32、FP16、BF16 等）；以及 GPU 硬件相关参数——目标架构编号、计算单元数量等。

在获取这些参数后，算子生成层首先根据输入特征图和卷积核的配置计算输出特征图的尺寸。随后，算子生成层会依据硬件架构特点和预设的优化策略，合理确定线程块和网格的划分方式，从而充分利用硬件的计算资源，形成最终的卷积运算操作。

万方数据

3.2.2 建模层

建模层是卷积算子实现的核心组成部分，承担着对卷积计算全流程进行精准建模与优化的任务。其主要作用是通过分层的方式全面描述卷积计算的算法过程，使其具备清晰的计算逻辑与可优化的执行路径。在这一层，设计特定的 Conv Dialect（卷积方言），将卷积计算的各个步骤进行分层拆解，确保计算逻辑清晰、可维护，并适用于不同硬件架构的优化需求。为了进一步提升卷积计算的性能，建模层在实现过程中引入了一系列优化策略如矩阵外积、双缓冲流水并行等优化策略，以提升卷积计算的性能。Conv Dialect 内部被进一步划分为三大类操作：软件抽象部分、硬件抽象部分和计算组件部分。软件抽象部分包括了卷积算法的高层次表示如前向卷积、反向数据卷积等；硬件抽象部分则涉及封装具体的底层指令；计算组件部分则是一些用于方便表达整体的逻辑算法过程的功能函数。

这一层的设计目的是使卷积计算过程能够以可移植且灵活的方式进行表达，并为后续的实现和执行层提供统一的计算描述基础。通过这种抽象方式，卷积算法不再依赖于具体硬件的实现细节，而是能够以一种普适的方式进行表达。这为后续的实现和优化提供了更大的灵活性，同时也为卷积算法在不同硬件平台上的迁移和优化奠定了基础。

3.2.3 实现层

实现层是卷积算法的具体实现层，主要负责将建模层所描述的卷积计算逻辑转化为接近硬件底层的高效计算实现。其核心目标是利用 MLIR 提供的通用组件，对建模层的逻辑表达进行重写，并最终生成适用于 GPU 端的中间表示。在 MLIR 框架下，实现层广泛使用 AffineDialect、GPUDialect、VectorDialect 等公共方言。其中 AffineDialect 主要用于处理循环优化，包括循环变换、数据流分析和存储优化，从而减少访存开销并提升数据局部性。GPUDialect 负责将计算逻辑映射到 GPU 计算单元，利用 GPU 线程层次结构（如 block、thread）来实现高效的并行计算。VectorDialect 则专注于向量化优化，充分利用 SIMD（Single Instruction Multiple Data）并行计算能力。向量化能够将多个标量计算合并为单个向量操作，从而减少指令开销，提高吞吐量。

万方数据

3.2.4 执行层

执行层是整个卷积算法实现过程中最底层的模块，直接与硬件交互，负责将卷积计算转化换为底层的硬件指令，确保卷积计算在特定硬件架构上高效执行。首先，执行层将卷积计算转换为 LLVM 中间表示（IR），然后通过 LLVM 的后端优化与代码生成机制，进一步映射为目标硬件的指令集，从而提升计算效率。LLVM 作为底层编译器基础设施，具备强大的代码优化、生成和硬件抽象能力，支持多种计算架构，包括 CPU、GPU 及专用加速器，使得同一卷积算法能够在不同硬件上运行，而无需进行大量适配。在具体执行过程中，执行层必须确保卷积算法的计算语义与上层保持一致，避免因转换误差导致计算错误。

3.3 Conv Dialect 设计

Conv Dialect 位于建模层，主要负责实现卷积在 GPU 端的计算逻辑。在 MLIR 中，dialect 可以视为一组操作（Operation，简称 Op）的集合，因此，Conv Dialect 的设计核心在于构建一系列能够有效描述卷积计算过程的 Op。这些 Op 需要具备良好的层次化结构，以便从高层算子到底层计算单元逐步映射，确保卷积计算逻辑能够被清晰表达。此外，这些 Op 需要支持矩阵外积、双缓冲流水并行等优化策略，从而确保卷积算子能够在硬件上高效执行。Conv Dialect 中的 Op 设计如图 3.2 所示包含三个部分：首先是描述软件计算任务的软件抽象部分，其次是描述硬件架构的硬件抽象部分，最后封装功能函数的计算组件部分。

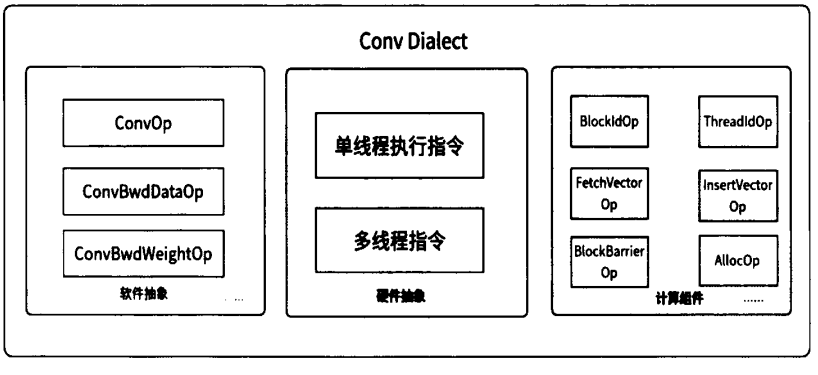


图 3.2 Conv Dialect 结构图

万方数据

3.3.1 软件抽象

Conv Dialect在软件抽象层面精心设计了一系列Op，以便清晰、高效地描述卷积计算的主要逻辑部分，这些 Op 主要包括前向卷积（ConvOp）、反向数据卷积（ConvBwdDataOp）以及反向权重卷积（ConvBwdWeightOp）。每个 Op 都具有明确的输入输出要求，通常包括卷积核、输入数据、输出数据等核心数据结构。

为了进一步提升卷积计算的效率，Conv Dialect 引入了基于 Implicit GEMM 算法的优化方法。Implicit GEMM 算法的核心思想是将卷积计算映射为标准的矩阵乘法问题，从而能够利用高效的矩阵运算库进行计算加速。在这一过程中，Conv Dialect 定义了多个 Op 来支持这一转化过程，其中包括 TransformOp 和 GemmOp。TransformOp 负责在逻辑上将输入数据从卷积的原始格式转换为适合矩阵乘法计算的大矩阵，这样做可以将卷积计算的多个维度映射为矩阵的行列结构，便于后续的矩阵运算。而 GemmOp 则是在逻辑层面上定义了标准的矩阵乘法计算过程，并且支持不同的计算精度和优化策略。

在优化卷积计算时，矩阵乘法作为核心计算单元，直接决定了整体计算性能。因此对矩阵乘法进行针对性的优化至关重要。为了充分发挥类 GPU 的架构优势，Conv Dialect 在设计时深入考虑了类 GPU 的硬件架构与内存层次结构，并通过精细化计算粒度来提升计算效率。具体而言，Conv Dialect 引入了 BlockGemmOp（线程块级矩阵乘）和 ThreadGemmOp（线程级矩阵乘）两个关键 Op，以支持不同层次的并行计算。其中，BlockGemmOp 主要在线程块级别执行矩阵运算，利用共享内存减少数据访问开销，而 ThreadGemmOp 则针对单个线程进行细粒度矩阵计算。为了进一步提高计算吞吐量，Conv Dialect 还采用了矩阵分块、双缓冲流水并行、寄存器优化等策略，对 GemmOp 进行分层式重写，使其在不同计算粒度下均能实现高效执行。

3.3.2 硬件抽象

为了更高效地执行卷积计算，Conv Dialect 提供了底层硬件指令抽象，旨在通过直接操作硬件资源来最大化计算性能。这些指令在更接近硬件的层面上定义了计算操作，以便实现更高的计算效率。这些底层硬件指令在形式上可以分为单线程执行指令和多线程协作指令。这两种指令抽象各自适应不同的硬件架构

万方数据

和计算需求，协同作用，确保卷积计算的高效执行。

单线程执行指令主要应用于类 CUDA 核心的硬件上执行，特点是每个指令在一个线程中执行，线程之间没有交互关系。这些指令的主要任务是执行较为基础的操作，比如内存访问、数据加载与转换等，它们直接控制线程对硬件资源的访问，影响整体的计算效率。单线程指令在实际应用中可能涉及多个操作，其中包括但不限于全局内存访存指令、共享内存访存指令、数据类型转换指令等。以全局内存访存为例，全局内存是 GPU 计算中的主要数据存储区域，不同的访存指令有着不同的性能特点。常见的访存指令包括 buffer_load_dword、buffer_load_dwordx2 等。buffer_load_dword 指令一次可以加载一个 32 位的数据，而 buffer_load_dwordx2 则一次可以加载两个 32 位的数据。通过选择合适的指令，可以优化内存访问的速度，减少内存访问的延迟。比如，如果需要连续加载 4 个 FP32（单精度浮点数）数据，可以选择连续调用 4 次 buffer_load_dword 指令，或者使用 buffer_load_dwordx4 向量化指令。相较于逐次加载，向量化指令能够并行加载多个数据元素，有效减少访存延迟，从而提升算子的整体计算效率。

与单线程执行指令不同，多线程指令是指在 tensor core 硬件核心上执行的一类指令（如 MMAC 指令），这些指令按照线程束级别进行调度与执行。一个线程束包含了 64 个线程，这些线程能够协同工作，共同完成某一计算任务。在张量核心上执行时，这些线程通过 SIMT（单指令多线程）架构以高度并行的方式执行任务，能够在每个时钟周期内进行更多的计算操作，因此相比于传统的单线程指令，多线程指令不仅能够减少调度开销，还能够更充分地利用硬件的计算资源，提高指令的执行效率。

在 Conv Dialect 中，设计了 BufferLoadOp、BufferStoreOp、MMacOp、FMAOp 等 OP 来封装这些底层硬件指令。

3.3.3 计算组件

为了在 Conv Dialect 中完整描述卷积过程，需要构建一系列组件 OP 来封装关键功能函数，从而在建模层更加清晰、高效地表达卷积的逻辑计算。这些组件 OP 不仅能降低代码冗余，还能提升整体模型的可维护性和可扩展性。以下是各组件及其功能的具体介绍：

万方数据

表 3.1 Conv Dialect 计算组件

组件	功能描述
BlockIdOp	得到当前的线程块 id
ThreadIdOp	得到当前线程 id
FetchVectorOp	获取连续的数
InsertVectorOp	插入连续的数
BlockBarrierOp	Block 级别的阻塞
AllocOp	内存分配
GlobalLoadOp	全局内存读取
GlobalStoreOp	全局内存贮存
InsertElementOp	插入元素
FetchElementOp	获取元素

3.4 下降通路

下降通路是 mlirGen 中的一个重要组成部分，主要负责将上层的计算表达逐步映射为底层硬件能够识别与高效执行的指令序列。本节主要讲解 Conv Dialect 内的 OP 下降以及 mlirGen 的方言下降。Conv Dialect 内的 OP 下降主要集中在对卷积操作的分解与优化上，其核心目的是将卷积算法调整为更适合 GPU 硬件架构的形式。在这一过程中会使用多种算法如矩阵分块算法、双缓冲流水并行算法等来优化卷积算法；mlirGen 的方言下降则是负责将建模层的中间表示下降转换成实现层的中间表示形式。

3.4.1 Conv Dialect 内的 OP 下降

由于卷积操作通常包括前向、反向和权重三种基本形式，为了实现代码的高复用性，并保持代码结构的清晰性和可维护性，选择在 Conv Dialect 中定义多种操作（OP），并通过这些操作将卷积计算逻辑进行拆解和实现如图 3.3 所示。这种设计不仅能够提高卷积运算的灵活性，还能让代码的整体结构更加清晰明了。此外，这种设计方式还使得将复杂的卷积运算转换为更底层的 Affine Dialect 变得更加简洁。

万方数据

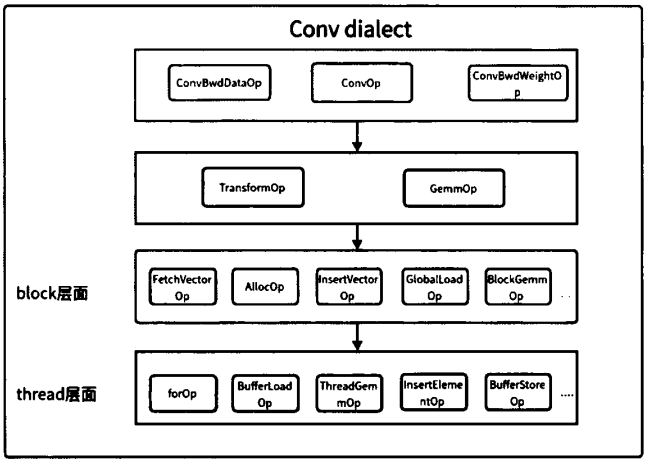


图 3.3 Conv Dialect 内的 Op 下降

以前向卷积（ConvOp）为例，详细解析其在 Conv Dialect 中的下降过程。前向卷积的实现依赖于 Implicit GEMM 算法，该算法通过将卷积运算转换为矩阵乘法来提升计算效率。因此，在下降过程中，ConvOp 首先会被转换为 TransformOp 和 GemmOp，以实现从卷积计算到矩阵乘法计算的映射。

TransformOp 的主要作用是在逻辑上根据卷积核的尺寸、步长等参数，将原始输入数据和卷积核重排，使其转化为适合矩阵乘法计算的形式。在具体实现过程中，核心问题在于如何高效地将输入数据和卷积核搬运到本地共享内存（LDS）。该过程需要解决多个核心问题。下降过程如图 3.4 所示，首先，需要建立线程索引与原始图像及卷积核具体位置的映射关系。其次，需要确定每个线程的搬运数据量，同时考虑 LDS 存储空间的限制，以及同一线程块（block）内线程的共享特性。因此需要先从 block 视角对 TransformOp 进行重新建模。具体来说，首先提取 TransformOp 中的参数，如卷积核的大小、步长，输入数据的大小、维度等。通过这些信息计算出 LDS 所需的内存大小，并确定每个线程需要搬运的数据量。然后使用 AllocOp 分配合适的 LDS 空间和寄存器空间，为后续的数据加载和计算做好准备。然后通过 GlobalLoadOp（全局内存读取）以向量化的形式加载原始输入数据到寄存器中。最后使用 FetchVectorOp 与 InsertVectorOp 逐元素提取数据，并存入 LDS。

在完成 block 级别的优化后，还需要进一步细化到 thread 级别，确保算子的

万方数据

完整下降。在此过程中，最关键的一步是对 GlobalLoadOp 进行优化降级，具体通过 BufferLoadOp 重写 GlobalLoadOp，以提升访存效率并优化线程间数据访问。

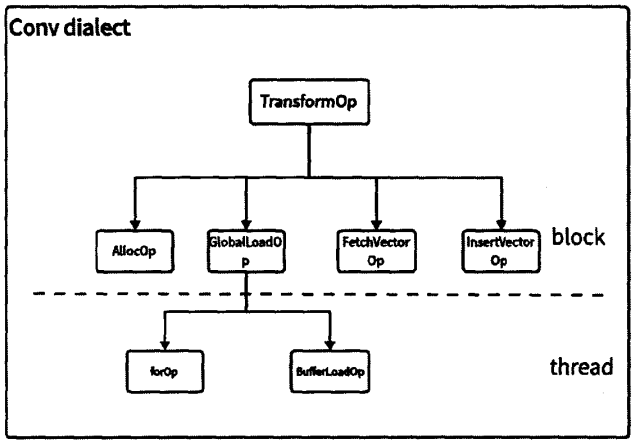


图 3.4 TransformOp 下降

在 GemmOp 的下降过程中，同样需要分别从 block 层面 和 thread 层面进行重新建模，以最大化计算资源利用率并提升整体计算效率。为了实现这一目标，同时结合矩阵分块和双缓冲流水并行等优化方法，Conv Dialect 特别引入了两个关键 Op：BlockGemmOp 和 ThreadGemmOp。下降过程如图 3.5 所示。

在 block 层面，会使用 BlockGemmOp 进行计算细化。BlockGemmOp 负责在 block 级别对矩阵乘法进行全局规划和任务分配。在这一层面，大规模矩阵乘法被拆分为多个较小的计算任务，每个任务被映射到不同的 block 进行独立执行，从而充分利用 GPU 的计算单元并行能力。具体而言，BlockGemmOp 计算每个 block 负责的小矩阵块，并通过 GlobalStoreOp 将计算结果存回全局内存。在此过程中，BlockBarrierOp 被用于 block 内部的同步，确保所有线程的计算任务在写回全局内存前同步完成，以避免数据竞争问题。此外，为了提升数据访问效率，每个 block 内部会进行数据预加载，利用 LDS 进行缓存，为下一轮计算做好准备，从而减少全局内存访问的开销。

在 thread 层面，需要对 BlockGemmOp 生成的任务进一步细化。首先，线程通过 FetchElementOp 和 InsertElementOp 操作，从共享内存加载计算所需的数据，并将其存入线程的寄存器中，以减少访存延迟。随后，ThreadGemmOp 负责执行

万方数据

线程级别的计算任务，每个线程计算部分矩阵乘法的结果，并累加到寄存器中。计算完成后，通过 block 级别的同步操作，确保所有线程的数据正确合并后，最终存入全局内存。

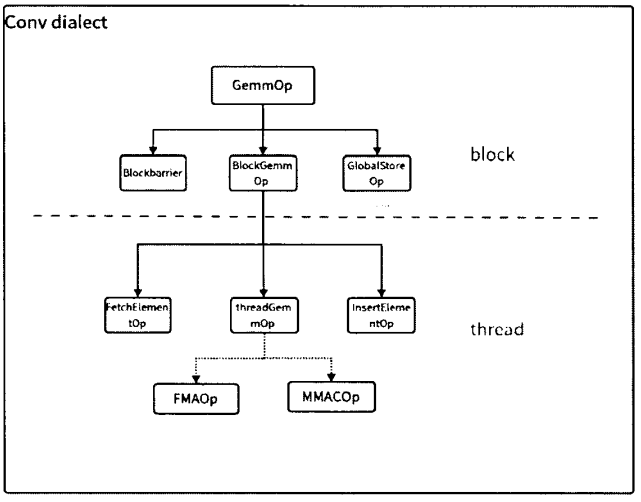


图 3.5 GemmOp 下降

3.4.2 Conv Dialect 下降

在 Conv Dialect 中，卷积算子的描述已经非常接近 GPU 端的实现形式。因此对于其下降过程，只需在底层 dialect 中重写与上层组件相对应的操作，以便将其高效地映射到 GPU 硬件上。下面重点介绍 FetchVectorOp 和 ThreadGemmOp 两个经典操作的下降过程。

FetchVectorOp 是用于从向量中提取元素的算子，在具体实现时需要根据目标类型进行判断。首先，需要确认目标类型是否为向量类型（vector::VectorType）。如果目标类型与源向量大小相同，操作可以简化为直接替换原有操作，直接用源向量的值替换目标向量，而无需进一步操作。这种简化处理能够减少不必要的计算，提升执行效率。如果目标向量的大小与源向量不同，使用 vector::ExtractElementOp 和 vector::InsertElementOp 这两个操作来重写其逻辑。

万方数据

算法3-1 FetchVectorOp 下降算法

```
Input: FetchVectorOp 操作
Output: vector算子表示形式
  if 目标类型为 vector::VectorType then
    if 目标向量与源向量大小相同 then
      replace 直接用源向量 Op 替换当前操作
    else
      size = getNumElements  // 获取目标向量的大小
      ret = createZeroConstantOp  // 创建一个零初始化的目标向量
      for i = 0 to size-1:
        cDest = arith::ConstantIndexOp(i)  // 目标索引
        cSrc = arith::AddIOp(base, cDest)  // 计算源向量索引 base + i
        v = vector::ExtractElementOp(源向量, cSrc)  // 提取源向量的元素
        ret = vector::InsertElementOp(v, ret, cDest)  // 插入到目标向量 ret
      end for
    end if
  else
    对于非 VectorType 类型，进行单元素提取和操作
  end if
```

ThreadGemmOp 用于实现线程级的矩阵乘法，首先，需要判断操作的数据类型。如果数据类型是整型，则采用传统的整数乘法与加法进行计算。如果数据类型是浮点型，则应使用浮点乘加运算（vector:: FMA）完成其逻辑。FMA 操作能够将乘法和加法合并为一个指令，减少浮点运算中的舍入误差，同时提高计算效率。

算法3-2 ThreadGemmOp下降算法

```
Input: ThreadGemmOp 操作（矩阵 A, B, C）
Output: vector算子表示形式作
    if 目标矩阵类型为 vector:VectorType then
        for i from 0 to gemmLoop:
            aVal = 从 A 读取值
            bVal = 从 B 读取值
            cVal = 从 C 读取值
            if 数据类型为 Integer then
```


万方数据

```
                mul = aVal * bVal   // 整数乘法
                result = mul + cVal // 加法
            else if 数据类型为 Float then
                result = vector:: FMA(aVal, bVal, cVal)   // 浮点乘加运算
            end if
            // 将计算结果存储回 bufferC
            存储 result 到 bufferC
        end for
    end if
```

3.5 本章小结

本章详细介绍了一个基于 MLIR 的卷积算子生成器 mlirGen 的设计。首先对 MLIR 的设计理念做了详细介绍，以此为基础阐述了 mlirGen 渐进式的分层架构，自顶向下分别是算子生成层、建模层、实现层、执行层。并依次对每层做了详细的介绍。然后对建模层的 Conv Dialect 做了详细的介绍。从软件抽象、硬件抽象、计算组件三部分介绍 Conv Dialect 中的 OP 设计。最后详细介绍了 mlirGen 的下降通路。首先介绍了 Conv Dialect 中 OP 的下降过程，以前向卷积的下降过程为例进行介绍，然后介绍了 Conv Dialect 的下降过程，重点介绍了 FetchVectorOp 与 ThreadGemmOp 的下降过程。

万方数据

4 基于优化算法和底层指令的优化通路设计

本章重点探讨了 mlirGen 的优化通路设计，旨在提升计算效率与资源利用率。优化策略融合了多种先进算法与底层指令，本章主要介绍矩阵外积技术、双缓冲流水并行算法以及 MMAC 指令。矩阵外积技术通过按外积方式计算矩阵的行列元素，大幅减少访存次数，从而降低内存带宽压力，有效优化矩阵乘法这一计算密集型任务。双缓冲技术采用两块缓冲区交替加载数据，使计算与数据传输并行执行，最大程度减少访存延迟，并提升数据流的利用效率。在此基础上通过循环展开形成流水进一步提高了指令级别的并行性。MMAC 指令是国产加速器专为矩阵乘法优化的硬件指令，通过 MMAC 指令可以提升卷积算子性能。

4.1 矩阵外积

卷积运算在经过 Implicit GEMM 算法的隐式转换后，其逻辑上被重新映射为可优化的二维 GEMM 矩阵乘法。在传统 GEMM 中，计算输出矩阵 C 的某个元素时，需要同时读取输入矩阵 A 的一整行和矩阵 B 的一整列。这种内积计算方式导致在求解 C 矩阵中相邻元素时，会重复加载 A 和 B 中的数据。如图 4.1 所示，计算 C 矩阵中一个 5×5 子块时，A 的每一行都被访问了 5 次，B 的每一列同样被访问了 5 次，导致在整个矩阵乘法过程中，A 的元素会被重复读取 N 次，而 B 的元素会被重复读取 M 次。这种反复读取 A 和 B 的方式不仅占用存储带宽，还增加了内存访问延迟，进而影响计算效率。特别是在大规模矩阵运算中，内存访问速度远低于计算速度，使得内存带宽成为系统性能的瓶颈。

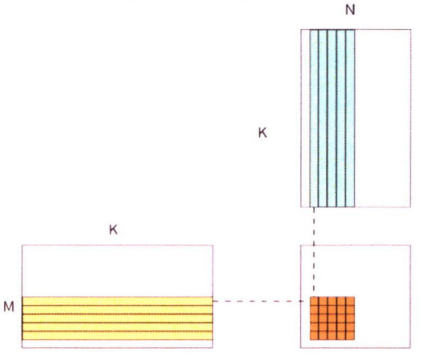


图 4.1 传统矩阵乘内积形式

万方数据

为了解决上述问题，可以采用矩阵外积的方法进行优化。如图 4.2 所示，矩阵 C 的计算过程被拆分为 K 个部分，每一部分通过读取 A 的一列和 B 的一行进行逐元素乘法，生成 C 矩阵的一个累加层，直至所有乘积结果累加完成。相较于传统的内积计算方式，矩阵外积显著减少了访存次数，避免了重复读取同一数据。此外，还可以将每层的累加中间结果存储在寄存器中，进一步降低存储带宽的消耗。该方法不仅提升了数据复用率，还充分利用了寄存器和高速缓存的优势，大幅提升矩阵运算的整体效率。通过将计算任务拆解为多个小规模的外积计算，数据在不同运算单元之间得以高效共享，从而减少内存访问延迟，提高并行处理能力，最终提升系统吞吐量。

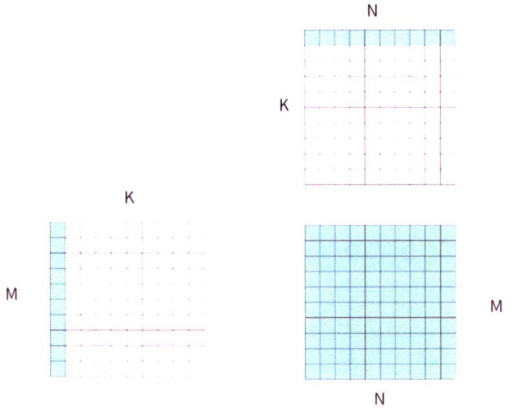


图 4.2 矩阵乘外积计算方式

4.2 双缓冲流水并行

4.2.1 串行执行机制

尽管卷积计算充分利用了国产加速器的多级存储体系，实现了数据在全局内存、共享内存和寄存器之间的高效流转，但其执行流程本质上仍然是“先取数、再计算”的串行模式。如图 4.3 所示：首先，大矩阵被划分为多个小块，每个 block 负责处理其中的一部分，并通过 block 内部的协作，将数据从全局内存加载到 LDS，以减少访存延迟。随后，这些 block 进一步拆分，每个 thread 提取自身对应的数据片段，并将其从共享内存搬运至寄存器，再使用 FMA（浮点乘加）指令完成计算工作。在这一个过程中，为了确保数据一致性，block 内部仍然需要

万方数据

进行两次同步（_syncthreads()）：第一次同步发生在数据从全局内存加载到共享内存后，以确保所有线程访问的数据已准备就绪；第二次同步则在寄存器计算完成后，使所有线程的计算任务同步结束，从而避免数据竞争问题。

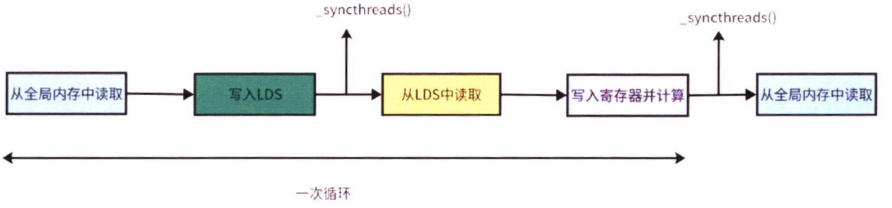


图 4.3 单次循环内线程执行流程

尽管串行执行机制能够确保计算过程的顺序性与精确性，但其固有的延迟问题可能会显著降低整体计算效率，尤其是在大规模矩阵运算中。这种延迟主要源于内存访问所引发的计算单元等待。在具体执行过程中，单个 block 在计算前需多次将 A 和 B 的子矩阵块加载至 LDS，以支持后续运算。然而，由于全局内存的访问速度远低于 LDS 和寄存器，每次数据搬运都会引入较长的等待时间，从而带来额外的访存开销并增加整体延迟。尽管 GPU 通过调度活跃 block 以隐藏全局内存访问的延迟，即在一个 block 等待数据加载的同时，其他 block 可执行计算任务，从而提升计算吞吐量，但在大规模矩阵计算中，由于 LDS 资源有限，可并行执行的 block 数量受到约束，使得访存延迟难以被完全消除，进而影响系统整体性能。此外，单个 thread 在执行计算前也需频繁从 LDS 提取数据并加载至寄存器，在此过程中，计算单元同样无法充分发挥作用，导致计算资源利用率降低。

4.2.2 双缓冲流水并行

为了降低串行执行机制带来的延迟，可以充分利用国产加速器的硬件特性，加速整个过程。由于国产加速器的计算单元和访存单元彼此独立，在单个线程内，只要不存在数据依赖关系，这两类操作便可并行执行，从而有效规避传统串行模式下计算与访存严格顺序执行的问题。这种并行机制能够显著缓解因数据迁移造成的访存延迟，特别是在涉及频繁数据交换和计算的任务中，可明显降低数据传输带来的额外开销。

万方数据

在卷积运算的串行执行过程中，每次计算任务与随后的访存操作相互独立，因此可以充分利用国产加速器的硬件特性，实现数据存取与计算的并行化。其核心思路是引入数据预取机制，使计算单元在执行当前计算的同时，预先加载下一轮计算所需的数据。这样，当新一轮计算开始时，数据已经就绪，计算单元能够立即投入计算，从而避免因数据加载延迟而导致的空闲等待。该机制通常采用双缓冲（Double Buffering）策略，即配置两个存储区域，使得在计算当前批次数据的同时，另一块缓冲区负责加载下一批数据，实现访存与计算任务的时间重叠。通过这一策略，可以有效减少计算单元的等待时间，从而提升整体执行效率。在具体实现中，可以在 LDS（本地数据共享内存）和寄存器中应用双缓冲技术，以进一步降低访存时延。接下来，将详细介绍在 LDS 中实现双缓冲算法的方法。

在将数据从全局内存加载到 LDS 的过程中，采用双缓冲策略，设计两块 LDS 存储区域，分别命名为 LDSA 和 LDSB。其中，LDSA 用于存放当前计算所需的数据块，而 LDSB 则用于预加载下一轮计算所需的数据块。如图 4.4 所示，在初始阶段（t0 时刻），程序首先从全局内存读取矩阵 A 和矩阵 B 的首个子矩阵，并将其存入 LDSA，完成数据的首次预取。随后，程序进入主计算循环。在 t1 时刻，线程从 LDSA 读取数据至寄存器进行计算的同时，矩阵 A 和 B 在 K 维度上的下一个子矩阵会从全局内存加载至 LDSB。通过这种方式，实现了计算与数据加载的并行化，减少了因数据等待导致的计算单元空闲时间。当 LDSA 中的数据计算完毕后，线程无需等待新数据加载，而是直接在 t2 时刻从 LDSB 中读取数据并继续计算。同时，LDSA 会启动新一轮的数据加载，将全局内存中的下一批数据搬运至共享内存。如此循环往复，计算和数据加载始终保持交替进行，直至主计算循环结束，从而有效提高计算效率。

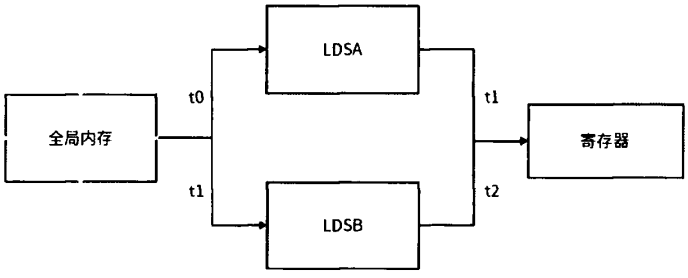


图 4.4 双 LDS 示意图

万方数据

在 LDS 与寄存器双缓冲的基础上，通过循环展开优化计算流程，并合理调度数据搬运与计算顺序，构建高效流水线，以提升硬件单元的并行度。图 4.5 展示了线程级循环流水。假设循环展开深度为 8，这意味着每个循环周期内需要执行 8 次计算指令。为确保计算与数据搬运的高效协同，在进入循环前，首先从 LDS 加载第一组计算数据至寄存器 A1，以准备后续流水线执行。主循环开始后，计算单元在对寄存器 A1 中的数据执行计算的同时，从 LDS 加载第二组数据至寄存器 A2。这种方式确保在当前数据处理完毕后，下一组数据已就绪，可立即投入计算。随着寄存器 A1 内数据计算完成，系统迅速将第三组数据加载至 A1，并同时对 A2 中的数据进行运算。整个过程通过交替加载与计算，使流水线始终保持连续运行状态，直至 8 组数据计算全部完成。这种数据加载与计算交替进行的策略，不仅极大地提升了硬件资源的利用率，还显著减少了由于等待数据而引起的延迟。流水线中各个阶段的任务可以重叠执行，实现数据传输与计算操作的无缝衔接。

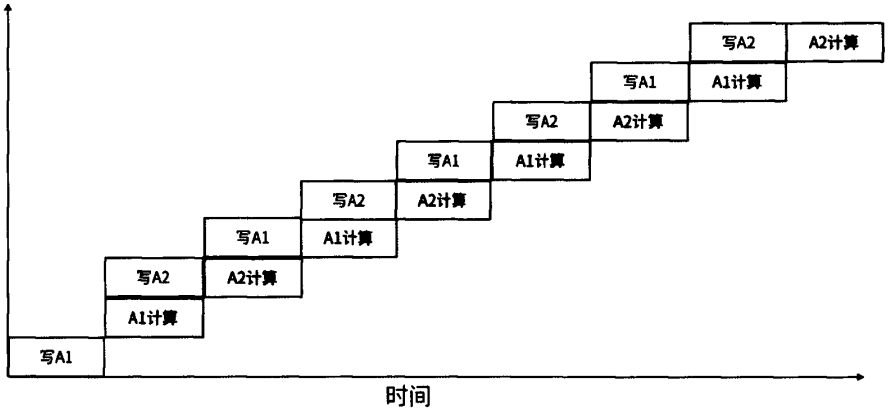


图 4.5 线程级循环流水并行

算法 4.1 双缓冲流水并行

Input：A[M][K], B[K][N]

Output：C[M][N]

根据线程块的划分计算需要的索引

执行 G_LOAD 并写入共享内存 LDSA

__syncthreads();

for numk to K/depth：

万方数据

对 LDSA 执行 LOAD 到寄存器组 A1、B1

对 A1、B1 执行 MAC 写入寄存器 C 并对 LDSA 执行 LOAD 到寄存器组 A2、B2

对 A2、B2 执行 MAC 写入寄存器 C 并对 LDSA 执行 LOAD 到寄存器组 A1、B1

重复以上过程

对 A2、B2 执行 MAC 写入寄存器 C

执行 G_LOAD 并写入共享内存 LDSB

__syncthreads()

end for

4.3 MMAC 指令

MMAC 指令是国产加速器为加速矩阵乘法而设计的一种专用硬件指令，它相对于传统的浮点乘加指令（FMA）在计算效率和硬件资源利用方面具有显著优势。传统的 FMA 指令是现代 GPU 中常用于矩阵乘法和其他浮点运算的基础指令，其基本操作是将两个浮点数相乘并加到累加器中。以 16×16 矩阵乘法为例，假设矩阵 k 的大小为 4，每个线程负责计算 2×2 的小矩阵块，由多个线程协作完成整个 16×16 矩阵的计算任务。在这种情况下，每个线程需要执行 4 次 FMA 指令来完成自己的计算部分。这种方式已经能提供较好的性能，但仍然存在一些局限性。首先，尽管 FMA 指令能够高效地进行浮点乘法和加法操作，但在计算密集型的任务中，它无法充分利用 GPU 的硬件资源。特别是当矩阵较大时，传统的 FMA 指令会导致指令执行频繁，增加了计算瓶颈，无法达到 GPU 硬件并行计算的最大性能潜力。此外，FMA 指令的执行过程中，仍然需要进行大量的内存访问和数据传输，这使得计算效率进一步受到制约。

为了克服以上这些问题，国产加速器引入了 MMAC 指令，它能够在单次执行中处理更大范围的矩阵数据，显著提高计算吞吐量。以 MMAC 指令中的 mmac_16x16x4_f32 为例，该指令能够在一次执行中计算 16×16 大小、深度为 4 的矩阵块。具体来说，指令的输入为 16×4 的矩阵 A 和 4×16 的矩阵 B，每次执行时，可以同时处理矩阵 A 的四行和矩阵 B 的四列的数据。图 4.6 展示了 mmac_16x16x4_f32 指令的具体计算过程。在实际执行过程中，线程 0 首先读取

万方数据

矩阵 A 中的 a0 元素和矩阵 B 中的 b0 元素，其他的计算数据则通过硬件自动获取，最终计算出矩阵 C 中的 c0、c4、c8 和 c12。与传统的 FMA 指令相比，MMAC 指令具有显著的优势。它通过硬件优化，使得每次执行中能够同时进行多个乘加运算，减少了指令的执行次数，提高了计算资源的使用效率，进而提高了矩阵乘法的计算吞吐量，为大规模矩阵运算和深度学习模型的训练提供了强有力的支持。

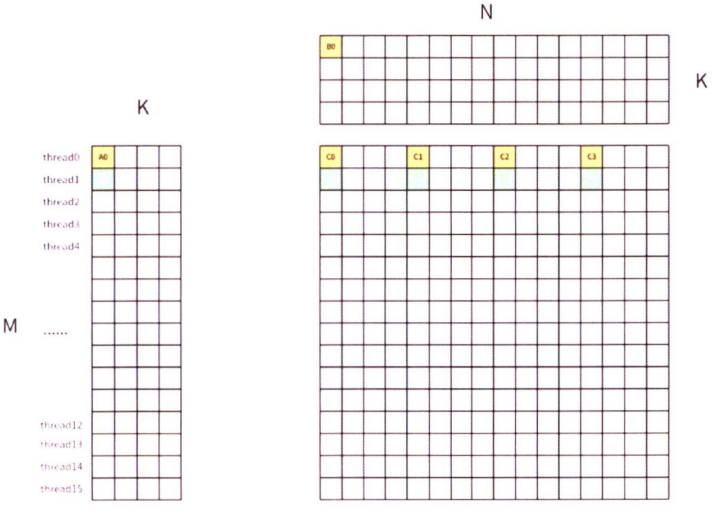


图 4.6 mmac_16x16x4_f32 计算过程

4.4 本章小结

本章主要对优化通路中的矩阵外积算法、双缓冲流水并行算法以及 MMAC 指令进行了详细介绍。矩阵外积算法通过调整矩阵乘法中的维度顺序，结合使用寄存器存储中间计算结果，显著减少了内存访问次数和写回次数。通过这种方式，内存带宽的需求得以降低，从而提升了整体计算效率。在此过程中，减少不必要的内存操作，不仅提高了计算速度，还减少了内存系统的负担，进一步优化了资源的利用；双缓冲流水并行算法则通过双缓冲技术有效优化了数据访问过程。该方法设立了两倍大小的 LDS 和寄存器，采用硬件存储与技术分块设计，能够有效掩盖访存延迟。通过这种设计，计算操作能够与数据访问并行进行，最大限度地减少了访存带来的性能瓶颈。此外，循环展开技术的引入进一步优化了流水线的效率，将循环操作拆分成多个更小的计算任务，降低了循环次数，并增强了指

万方数据

令的并行性。MMAC 指令是国产加速器专为矩阵乘法优化的专用硬件指令，基于线程束进行矩阵块计算，有效的减少了指令执行次数，提高计算资源的利用率，从而大幅提升矩阵乘法的计算吞吐量。这些优化措施协同作用，显著提升了计算过程的并行度和整体性能。

万方数据

5 实验结果与性能评估

5.1 实验环境及测试用例

本章的实验是在前文介绍的国产加速器上进行的。加速器采用类似 GPU 的架构，具有 64 个计算单元，总计 4096 个流处理器，核心频率达到 1443 MHz。此外，每个加速器还配备了 16GB 的 HBM2 全局内存，并且拥有 64KB 的共享内存，支持高带宽的数据传输。该平台的计算性能非常强大。具体来说，在双精度计算方面，其峰值性能可以达到 5.9 TFLOPS，而在单精度计算时，峰值性能则提升至 11.8 TFLOPS。

为了全面评估 mlirGen 的性能，实验不仅仅限于对比国产加速器上的手写算子库的表现，还特别将其与 NVIDIA V100 平台上的 cuDNN 库中的卷积手写算子进行了对比。NVIDIA V100 采用了 Volta 架构，并且搭载了 32GB 的 HBM2 设备内存，拥有 80 个计算单元以及 5120 个 CUDA 核心。这些硬件特点使得 V100 在计算密集型任务中展现了强大的性能。在计算能力方面，V100 的单精度计算峰值性能达到了 15.7 TFLOPS，双精度计算峰值性能则为 7.8 TFLOPS。国产加速器和 NVIDIA V100 平台的硬件参数对比如表 5.1 所示。

表 5.1 硬件参数对比

	国产加速器	NVIDIA V100
时钟心频率	1319Mhz	1380Mhz
显存带宽	800GB/s	900GB/s
全局内存	16GB HBM2	32GB HBM2
共享内存大小	64KB	96KB
微处理器数	64	80

实验中所选取的卷积参数来自多个广泛应用的卷积神经网络，如 ResNet50 和 YOLO 等。这些网络代表了不同的计算需求和网络结构，涵盖了从图像分类到目标检测等多种典型应用场景。通过对这些网络的测试，可以更全面地验证 mlirGen 在多种应用环境下的稳定性与高效性，确保其计算性能与主流技术保持一致，从而满足实际应用的性能需求。下表 5.2 详细列出了实验中卷积层的参数组合形式。N 代表输入图像的数量，C 表示图像的通道数，H 和 W 则分别是图

万方数据

像的高度和宽度。K 代表卷积核的数量，而 R 和 S 则是卷积核的高度和宽度。此外，Pad_H 和 Pad_W 表示图像的边缘补零大小，Stride_H 和 Stride_W 则定义了卷积核在图像上滑动的步长。

表 5.2 输入参数形式

输入参数	含义
N，H，W	输入特征图的图片数、高、宽
K，C	卷积核的数量与图像的通道数
R，S	卷积核的高、宽
Pad_H，Pad_W	输入图像的边缘补零
Stride_H，Stride_W	卷积核在图像上滑动的步长

5.2 实验校验标准

在相同精度和参数设置下，对不同加速器的算子性能进行评估时，仅依赖运行时间作为衡量标准并不够全面。这是因为不同加速器在硬件架构上存在显著差异，例如工作频率、处理核心数量和内存带宽等，这些因素都会直接影响计算性能。例如，较高的工作频率通常意味着单个核心在单位时间内能够执行更多操作，从而提升计算效率；而更多的处理核心则能够并行处理更多任务，提高整体计算能力。因此，为了更全面、科学地评估不同加速器的性能，特别是在计算性能与能耗之间的权衡，必须引入“能效比”这一指标。能效比不仅衡量了加速器的计算能力，还将能耗因素纳入考量，使性能分析更加全面。其计算公式如下：

$$radio = \frac{Time0 \times TFLOPS0}{Time1 \times TFLOPS1} \tag{5.1}$$

在上述公式中，*Time0* 和 *Time1* 是分别代表不同加速器在相同精度下执行相同计算任务所需的运行时间，而 *TFLOPS0* 和 *TFLOPS1* 则是不同加速器在相同精度下的理论峰值性能。在与 NVIDIA V100 加速器的对比实验中，能效比被作为核心评估指标，能够更准确地反映各加速器在实际计算任务中的表现。通过计算不同加速器的理论峰值性能，并结合其实际执行时间，可以得出每款加速器的能效比。对比这些能效比，不仅能够直观展现不同加速器在相同计算任务下的性能差异，还能衡量它们的能源利用效率。较高的能效比意味着该加速器能够在更低的能耗下提供更优的计算性能，从而展现更强的综合优势。

万方数据

5.3 前向卷积实验

5.3.1 Fp32 测试

为了全面评估算子的性能，实验选取 100 组具有代表性的输入参数作为测试样例。这些参数涵盖了不同规模的卷积神经网络及多种输入数据分布，确保实验结果的广泛适用性和可靠性。图 5.1 直观展示了 mlirGen 与国产加速器上手写卷积算子的平均耗时对比情况。图中的横轴表示不同的输入参数组，纵轴则反映了相应情况下的平均计算耗时。实验结果表明，mlirGen 的性能平均达到国产官方手写算子库的 85%，这表明 mlirGen 自动化生成的卷积算子能够在计算性能上接近手写实现，基本满足工业需求。值得注意的是，在处理部分小规模输入时，mlirGen 的表现尤为突出，不仅能够接近手写算子库的性能，在某些情况下甚至超越手写实现。

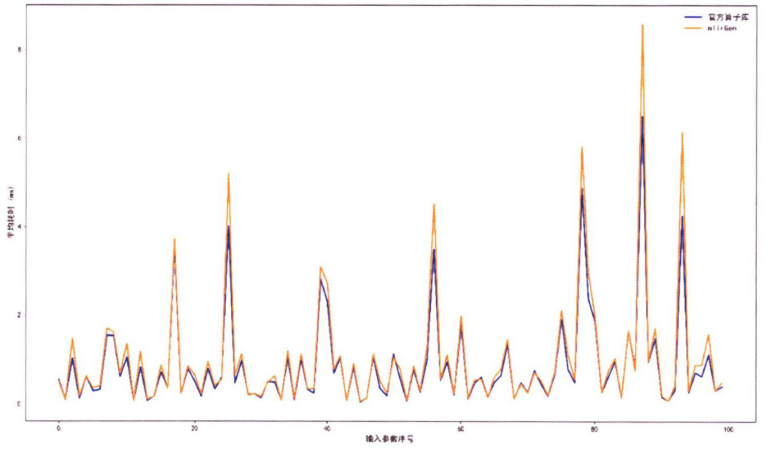


图 5.1 FP32 精度下 mlirGen 与官方算子库前向卷积性能对比

在与 V100 平台的 FP32 精度算子进行性能对比时，首先需要在两种硬件平台上针对每组测试参数进行多次实验，并统计其平均执行时间。随后，结合两种平台的理论峰值性能，通过公式 5.1 得出自研算子与 V100 平台算子之间的能效比。图 5.2 展示了不同输入参数对应的能效比，其中横坐标表示各测试参数组，纵坐标则直观呈现相应的能效比数据。对比结果显示，约 80%的参数组能效比大于 1，这表明在相同计算负载下，mlirGen 生成的算子相比 V100 平台算子能够

万方数据

更高效地利用硬件资源。这一性能优势对于大规模工业应用至关重要。在实际的工业应用中，计算资源的合理配置和高效利用直接影响到系统的整体运行效率，尤其是在面对复杂计算任务和大规模数据处理时，硬件资源的浪费将带来不可忽视的成本。通过提高硬件资源的利用率，mlirGen 能够有效减少计算过程中的资源浪费，从而显著降低整体的计算成本。

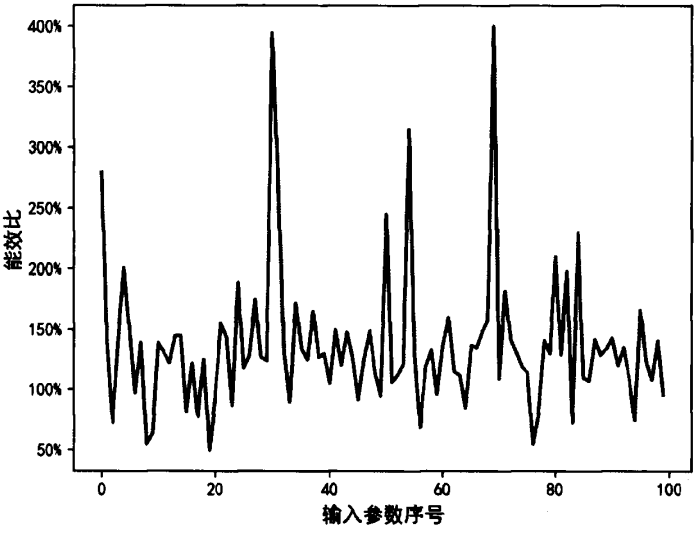


图 5.2 FP32 精度下 mlirGen 与 V100 平台前向卷积算子能效比

5.3.2 Fp16 测试

为了全面评估算子的性能，除了在 Fp32 精度下进行详细测试外，针对 Fp16 类型的测试也同样至关重要。与 Fp32 相比，Fp16 具有许多显著优势，尤其是在内存占用、计算带宽以及计算效率方面。由于每个数仅占用 16 个比特，Fp16 在内存中的存储开销是 Fp32 的二分之一，这导致使用 Fp16 能够在相同的计算资源下处理更多的数据，尤其适用于深度学习模型训练和推理中对高吞吐量和低延迟的需求。因此，许多现代硬件平台，尤其是异构加速平台，已广泛支持 Fp16 计算模式，并在相应领域取得了显著的应用成果。

为了全面评估 Fp16 算子的性能，同样选择了 100 组不同的输入参数作为测试用例。测试结果通过图 5.3 进行了清晰展示，其中横轴代表每组特定的输入参数，纵坐标则表示在该输入参数下的平均耗时。通过对比图中的数据，可以直观地看出，mlirGen 生成的算子在 Fp16 下仅有国产官方手写算子库性能的 70%。

万方数据

这一结果表明，尽管 mlirGen 在 Fp16 下有一定的优势，但与传统的卷积手写算子相比，仍存在明显的性能差距。深入分析后发现，造成这一性能差距的主要原因在于，现有的官方算子库在 Fp16 卷积算子的核心计算部分和写回部分进行了深度的专门优化。具体来说，官方算子库通过采用更高效的内存访问模式和数据重排策略，能够最大化硬件的并行处理能力，减少内存带宽的消耗，从而显著提升计算效率。相比之下，mlirGen 在 Fp16 上的实现尚未完全覆盖这些底层优化策略，尤其是在数据搬运和内存访问方面，导致了在低精度计算场景下的性能差距更加显著。

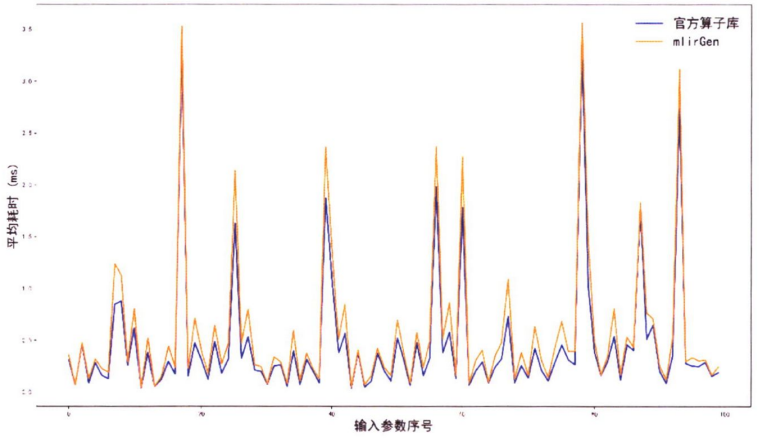


图 5.3 FP16 精度下 mlirGen 与官方算子库前向卷积性能对比

虽然 mlirGen 在 FP16 下性能难以与经过深度优化的手写算子库相匹敌，但在算子的时间成本和维护成本上仍有一定优势。而且它能够在不依赖复杂人工优化的情况下快速适应不同硬件平台。这使得 mlirGen 在某些场景下仍然具有较高的实用价值，尤其在需要进行快速开发和迭代的项目中，能够有效节省开发和维护时间。

从图 5.4 对比 V100 的 Fp16 测试结果可以看出 mlirGen 在能效比上依然有优势，有 73%的算子性能超过 1。因此，尽管目前在直接性能对比中，mlirGen 的 Fp16 算子未能完全匹配手写优化算子，但随着对生成策略和底层优化技术的进一步提升，mlirGen 无疑有巨大的潜力可以挖掘。通过对这一生成框架的持续优化，我们相信其能够在未来的深度学习应用中提供更为出色的性能表现。

万方数据

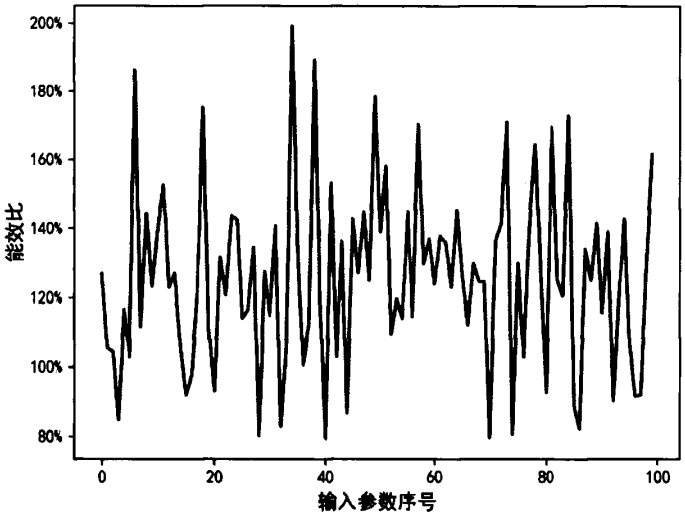


图 5.4 FP16 精度下 mlirGen 与 V100 平台前向卷积算子能效比

5.4 反向数据卷积实验

本节主要展示了 FP32 精度下 mlirGen 在反向数据卷积操作中的性能对比实验，旨在全方位评估 mlirGen 的计算性能，并进一步与国产手写算子库进行对比，以突出 mlirGen 在实际应用中的优势。为了确保评估的全面性和代表性，实验选取了多个不同规模的卷积神经网络，并从中随机抽取了一系列参数进行测试。这些参数涵盖了不同的网络层数、通道数、输入尺寸以及卷积核大小，力求尽可能模拟真实场景下的计算负载。

实验结果如图 5.5 所示，展示了 mlirGen 与国产手写算子库在 FP32 精度下的反向数据卷积性能对比。从结果分析来看，mlirGen 在 FP32 精度下的反向卷积计算性能达到了国产手写算子库的 82%，相较于前向卷积，其性能出现了一定程度的下降。尤其是在某些特定的卷积核尺寸下，这一差距更为明显。

万方数据

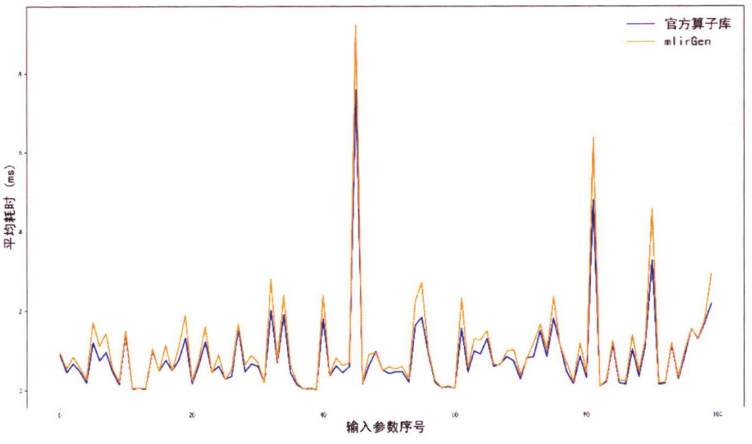


图 5.5 FP32 精度下 mlirGen 与官方算子库反向数据卷积性能对比

通过对实验参数进行详细分析，发现性能差距的一个主要原因在于 3×3 卷积核的反向卷积性能较为逊色。这是因为国产手写算子库在处理 3×3 卷积核时采用了 Winograd 算法，而 mlirGen 并没有充分利用这一算法。Winograd 算法是一种高效的卷积计算方法，尤其适用于小卷积核（如 3×3）的场景。它通过数学变换将卷积计算转换为更少的乘法运算，从而显著降低了计算复杂度，尤其在小卷积核的应用中，Winograd 算法表现出极大的优势。具体来说，Winograd 算法在卷积过程中将输入数据进行变换，利用数学公式将卷积计算拆解成低维矩阵的乘法。这种转化方式能够有效减少在小卷积核情况下的冗余计算。因此在处理 3×3 卷积核时，Winograd 算法能够充分利用其优化特性实现比传统的 Implicit GEMM 方法更低的计算成本，尤其在卷积操作涉及多个通道或较大输入图像时，Winograd 算法的优势更加明显。

尽管在 3×3 卷积核大小的计算任务上，mlirGen 的性能相较于某些优化方案略显逊色，但在处理其他尺寸的卷积核时，mlirGen 依然能够保持较为稳定且高效的表现。特别是在较小卷积核的计算场景中，mlirGen 凭借其灵活的计算调度策略，展现出了良好的适应性和优化效果。从整体性能表现来看，mlirGen 在反向数据卷积计算中的表现与前向卷积大体一致，依然展现了较强的竞争力。这说明无论是在前向传播还是反向传播的计算过程中，mlirGen 都能够较好地发挥其优化能力。

万方数据

此外，图 5.5 展示了 mlirGen 与 V100 算子在能效比方面的对比实验。从图中可以看出，mlirGen 在能效比上的表现依旧保持着一定优势，有 77%的算子的能效比性能达到 1 以上。

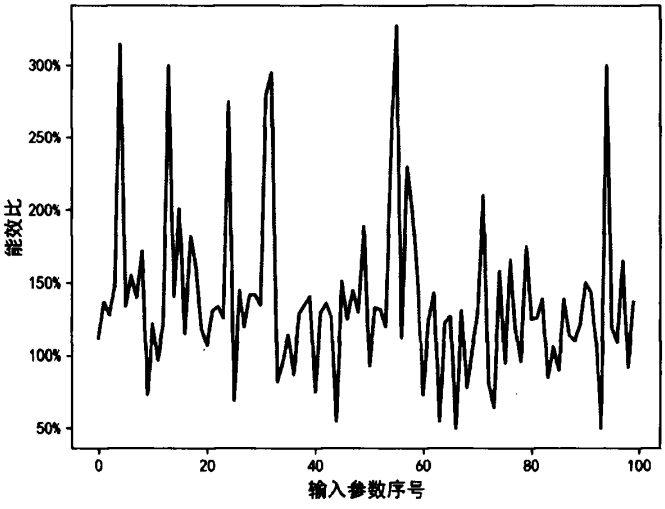


图 5.6 mlirGen 与 V100 平台反向数据卷积算子能效比

5.5 本章小结

本章主要针对 mlirGen 的性能进行了深入评估。首先介绍了实验环境的配置包括硬件平台的结构和算力信息等。然后阐述测试用例的具体形式主要涵盖了常见的卷积算子前向与反向计算，并涉及到不同数据类型（FP32 和 FP16）的测试。实验过程中，首先对 FP32 和 FP16 的卷积前向算子进行测试并统计平均耗时，通过与国产算子库原有算子和 NVIDIA V100 平台上的算子进行性能对比。结果显示在 FP32 精度下，mlirGen 的性能大体上满足工业界对卷积算子的需求，能够在大多数场景下稳定运行，并且具有较好的执行效率。而在 FP16 精度下，尽管 mlirGen 的性能稍逊于 FP32 的表现，但其在能效比上的优势仍然不容忽视。最后对反向数据进行测试，结果于前向卷积性能大体一致。

万方数据

6 总结与展望

6.1 工作总结

本文主要聚焦于结合 MLIR 的灵活性与国产加速器的硬件特性，设计并实现了一款高效的 MLIR 卷积算子生成器——mlirGen，专门用于在国产加速器上生成和优化卷积算子。mlirGen 采用分层架构，充分利用 MLIR 的多层次表示能力，支持不同抽象层级的优化，使得卷积算子的表示、转换和优化更加系统化且可扩展。在框架设计中，构建了一条合理的优化下降通路，确保从高层表示到底层硬件指令的映射过程中，能够逐步应用多种优化策略，以提升计算效率。在 mlirGen 中，特别设计了 Conv Dialect，用于精确描述卷积算子在国产加速器上的计算逻辑。此外，mlirGen 还在优化通路中集成了矩阵外积计算、双缓冲流水并行等优化技术，并结合国产加速器的 MMAC 等底层指令，充分挖掘硬件的计算潜力，以显著提升整体性能。mlirGen 旨在解决传统手写卷积算子在优化成本高、跨平台移植复杂、硬件适配重复等方面的挑战，还能为国产加速器的算子优化提供创新的解决方案。全文工作总结如下：

(1) 基于 MLIR 构建了一套分层卷积算子生成框架，该框架分为：算子生成层、建模层、实现层和执行层。充分发挥了 MLIR 在灵活性和多层次表示上的优势。在框架设计中，采用了合理的下降通路，包括 Conv Dialect 内 OP 的下降和 Conv Dialect 的下降以实现了不同粒度的优化。通过这些设计使得高层次的算法可以在保证灵活性的前提下，通过逐步优化转化为硬件能够理解和执行的指令序列，从而有效利用国产加速器的并行计算能力，实现高效的卷积运算。

(2) 基于 Implicit GEMM 算法，设计了 Conv Dialect，以实现卷积在国产加速器上的计算逻辑。该设计充分考虑了卷积计算的复杂性和层次结构，通过从软件抽象、硬件抽象和组件设计三个方面，构建了卷积计算的运算。具体而言，Conv Dialect 通过分层次的方式，能够在 block 层面和 thread 层面精确描述卷积计算过程。这种分层结构使得在建模层能够生成类似 GPU 端的代码，从而便于将卷积计算表达逐步映射到公共 Dialect 中，最终实现高效的硬件执行。

(3) 基于矩阵外积、双缓冲的循环流水并行策略等算法并结合 MMAC 等底层指令，设计优化通路，旨在提升计算效率和减少系统瓶颈。矩阵乘法是 Implicit

万方数据

GEMM 算法的核心操作，通过将矩阵的行列元素按外积方式进行计算，不仅降低了访存次数，还减少了计算过程中对内存的频繁写回操作，从而有效减少了内存带宽的消耗。传统的矩阵乘法算法往往会因为频繁的内存访问和写回导致性能瓶颈，而通过采用外积方式，可以减少重复计算和数据传输的开销，从而提升整体计算效率。此外，为了减小串行执行机制下访存延迟对性能的影响，采用了双缓冲流水并行算法。该方法充分发挥国产加速器的并行访存与计算能力，通过双缓冲机制实现数据加载与计算的时间重叠。两块缓冲区交替工作，使数据传输与计算能够同步进行，从而降低访存瓶颈。整个计算过程中，引入循环展开技术，以减少循环次数、提升指令并行度，最终形成高效的循环流水并行执行模式。最后，针对 FMA 指令的处理大矩阵时的频繁使用导致的计算瓶颈问题采用 MMAC 指令来优化底层矩阵乘法。MMAC 以线程束为单位来执行矩阵运算，使得每次执行中能够同时进行多个乘加运算，减少了指令的执行次数，提高了计算资源的使用效率，进而提高了矩阵乘法的计算吞吐量。这些优化策略结合使用，不仅显著提升了卷积运算的性能，还充分发挥了硬件资源的并行计算能力。

最后，在国产加速器上对卷积算子生成器的前向卷积和反向数据卷积进行了性能测试。实验共设置两组对照实验：第一组以国产加速器的自研手写算子作为基准，第二组选取 NVIDIA V100 平台上的 cuDNN 库算子进行对比，并以能效比作为跨平台性能评估标准。实验从不同规模的卷积神经网络中抽取参数，以确保测试结果的广泛适用性和可靠性。随后，在 FP16 和 FP32 精度下，分别统计 mlirGen、国产手写算子库及 cuDNN 库在相同测试用例下的平均运行时间，并计算 mlirGen 算子相较于 V100 平台算子的能效比。

最终的性能对比分析结果表明，在前向卷积测试中，生成器在 FP32 精度下的性能达到国产手写算子库的 85%，在 FP16 精度下达到 70%。与 V100 平台算子相比，FP32 精度下 80%的算子能效比超过 1，FP16 精度下 73%的算子能效比超过 1。在反向数据卷积测试中，生成器在 FP32 精度下的性能达到国产手写算子库的 82%，且整体能效比相较 V100 平台算子维持在 77%，总体性能与前向测试保持一致。实验结果表明，mlirGen 能够高效生成符合工业需求的卷积算子，在保证计算精度的同时，具备较高的性能和能效比。相比手写算子库，mlirGen 在部分场景下实现了接近手工优化的性能，同时提供了更高的开发效率和可维护性。此外，与 V100 平台相比，mlirGen 在多个算子配置下展现出能效优势，尤其在 FP32 精度下，部分算子能效比超过 1.2，表明在特定计算场景下，其优

万方数据

化策略能够有效发挥硬件潜力。综合来看，mlirGen 在性能、能效比和开发便捷性方面均展现出较强的竞争力，为自动化算子生成提供了一种可行的高效方案。

6.2 展望

本文基于结合 MLIR 和国产加速器的性能设计卷积算子生成器——mlirGen。通过设计多种针对性优化算法，提升了整体性能。该生成器能够高效生成符合工业需求的卷积算子，且在能效比方面，表现出显著优势。这不仅证明了设立策略和优化算法的有效性，也充分展示了 MLIR 在代码生成的巨大优势的巨大潜力。针对本文的优化工作，仍有以下研究方向可以继续开展：

(1) mlirGen 在 FP16 精度下的性能表现不佳。未来可从优化内存访问模式、结合硬件架构特性调整数据重排方式等方式来优化，以便更好地适配 FP16 数据格式，从而提升计算效率。此外还可以根据热点参数做一些优化特化，进一步优化整体性能。

(2) mlirGen 在处理 3×3 卷积核的反向数据卷积时性能较差，未来可以考虑使用 Winograd 算法对反向数据卷积进行特化优化。通过在卷积算子生成器中集成这一优化策略，可以有效降低计算复杂度，提升反向数据卷积的整体性能。

万方数据

参考文献

[1] Iván F S,Edward Y,Marta A,et al.Convolutional neural networks to identify malformations of cortical development: A feasibility study[J].Seizure: European Journal of Epilepsy,2021,9181-90.

[2] Higaki S,Matsui Y,Miwa M,et al.Leveraging computer vision-based pose estimation technique in dairy cows for objective mobility analysis and scoring system[J].Computers and Electronics in Agriculture,2024,217:108573.

[3] Meidan D,Aiping Q,Haiqin Z,et al.An enhanced vision transformer with wavelet position embedding for histopathological image classification[J].Pattern Recognition,2023,140:109532.

[4] Xiwei YZhixin L,Xinfang Z,et al.Mining graph-based dynamic relationships for object detection[J].Engineering Applications of Artificial Intelligence,2023,126:106928.

[5] Kim Y.Convolutional Neural Networks for Sentence Classification[C]//Proceedings of Empirical Methods Natural Language Processing (EMNLP). 2014:1746-1751.

[6] Prilop N, Rotsaert C, Vanderlinde R. Fostering pre-service teachers' classroom management knowledge, self-efficacy, and professional vision: The effect of different expert feedback and pre-service teachers' feedback perceptions during online video analysis[J]. Teaching and Teacher Education, 2025, 157:104949-104949.

[7] WeiTing H,Jennifer W,Jin X.A Natural Language Processing–Driven Framework for Policymaking in Infrastructure Development[J].Journal of Construction Engineering and Management,2025,151(5):04024034.

[8] Kumar A,Yadav P S,Kumar A.An improved feature extraction algorithm for robust Swin Transformer model in high-dimensional medical image analysis.[J].Computers in biology and medicine,2025,188:109822.

[9] Kim C Y,Choi H J,Sugawa T .Coefficient bounds and convolution properties for certain classes of close-to-convex functions[J].Proceedings of the Japan Academy, Series A, Mathematical Sciences,2000,76(6):95-98.

[10] Koppers S,Merhof D .DELIMIT PyTorch - An extension for Deep Learning in Diffusion Imaging.[J].CoRR,2018,abs/1808.01517.

[11] Trademarks; An Application for the Trademark "TENSORFLOW" Has Been Filed by Google[J].Information Technology Newsweekly,2015.

[12] Lattner C, Amini M, Bondhugula U, et al. MLIR: A compiler infrastructure for the end of moore's law[J]. arXiv preprint arXiv:2002.11054, 2020.

[13] Lattner C , Amini M , Bondhugula U ,et al. MLIR: Scaling Compiler Infrastructure forDomain Specific Computation[C]//2021 IEEE/ACM International Symposium on CodeGeneration and Optimization (CGO).ACM, 2021,2-14.

[14] Semystiaha V .The role and place of secret collaborators in the informational activity of the

万方数据

GPU-NKVD in the 1920s and 1930s[J].Cahiers du monde russe,2001,42(42/2-4):231-244.

[15] Bazzi S H,Jaber A R,Hajj E M A , et al.Enhanced CPU Design for SDN Controller [J].Micromachines,2024,15(8):997-997

[16] Yuchen J ,Qiyu W ,Xuqing W , et al.FPGA-accelerated deep neural network for real-time inversion of geosteering data[J].Geoenergy Science and Engineering,2023,224:211610.

[17] Solans M J,Børve A,Hejnol A ,et al.Diarylamidine activation of a brachiopod DEG/ENaC/ASIC channel.[J].The Journal of biological chemistry,2024,301(1):108066.

[18] 曹义魁,陆忠华,张鉴,等.面向国产加速器的CFD核心算法并行优化[J].数据与计算发展前沿,2021,3(04):93-103.

[19] Bai N ,Chen X ,Wang H , et al.High-sensitivity piezoresistive sensors based on functionalized carbon nanotube/TPU composite for human motion detection[J].Polymer,2025,322128156-128156.

[20] Krill P .Mojo language marries Python and MLIR for AI development[J].InfoWorld.com, 2023-05-04.

[21] Bondhugula U .High Performance Code Generation in MLIR: An Early Case Study with GEMM[J].arXiv preprint arXiv:2003.00532.

[22] Katel N , Khandelwal V , Bondhugula U .High Performance GPU Code Generation for Matrix-Matrix Multiplication using MLIR: Some Early Results[J].arXiv e-prints arXiv:2108.13191, 2021.

[23] Ragoomundun N ,Beeharry G .A cuBLAS-based GPU correlation engine for a low-frequency radio telescope[J].Astronomy and Computing, 2020, 32:100407.

[24] Barthels H , Bientinesi P, Copik M, Grosser T. MOM: Matrix Operations in MLIR[J]. arXiv preprint arXiv :2208.10391, 2022.

[25] Thangamani A , Jost, Loechner V , Genaud S. Lifting code generation of cardiac physiology simulation to novel compiler technology[J]. Proceedings of the 21st ACM/IEEEInternational Symposium on Code Generation and Optimization (CGO 2023), 2023, 68–80.

[26] Kumar A , Khedkar A , Shrivastava A .DSP-MLIR: A MLIR Dialect for Digital Signal Processing[J]. arXiv preprint arXiv:2408.11205,2024.

[27] Henningsen G .Big Data for the Prediction of Forced Displacement[J].International Migration Review,2025,59(1):141-164.

[28] Ye G P ,Li Z ,Yang Z , et al.Periodic watermarking for copyright protection of large language models in cloud computing security[J].Computer Standards & Interfaces,2025,94103983-103983.

[29] Ramachandran S ,Jayalal M ,Vasudevan M , et al.Combining Machine Learning techniques and Genetic Algorithm for predicting run times of High Performance Computing jobs[J].Applied Soft Computing,2024,165112053-112053.

[30] Mohamed A ,Hossam H ,K. R C , et al.PV-Net: An innovative deep learning approach for efficient forecasting of short-term photovoltaic energy production[J].Journal of Cleaner Production,2021,303:127037.

万方数据

[31] Iwasawa M ,Namekata D ,Nomura K , et al.Extreme-scale particle-based simulations on advanced HPC platforms:Lessons from PEZY-SC2, Sunway Taihulight and NVIDIA Volta[J].CCF Transactions on High Performance Computing,2020,2(7):183-195.

[32] Kohei F ,Takuma Y ,Yuma K , et al.Calculation of cross-correlation function accelerated by TensorFloat-32 Tensor Core operations on NVIDIA's Ampere and Hopper GPUs[J].Journal of Computational Science,2023,68:101986.

[33] Initial Two CoreWeave Data Centers Hosting NVIDIA Hopper GPUs Now Operational in the UK[J].Telecomworldwire,2025.

[34] IonQ Highlights New Chemistry Application and Nvidia CUDA-Q[J].Manufacturing Close - Up,2024.

[35] AMD and ECARX to Collaborate on Immersive Digital Cockpit In-Vehicle Computing Platform for Next-Generation Electric Vehicles; Advanced digital cockpit will be powered by AMD Ryzen™ Embedded V2000 Processor and AMD Radeon™ RX 6000 Series GPUs[J].M2 Presswire,2022.

[36] Meena V ,Barath S ,Singh S , et al.Role of Imaging Spectrum Along With Other Diagnostic Modalities in Rhino-Orbital-Cerebral Mucormycosis (ROCM).[J].Cureus,2024,16(2):e53962-e53962.

[37] 符鹤,谢永芳.基于国产化图形芯片JM5400 的座舱显示系统设计[J].计算机工程与科学,2016,38(10):2083-2090.

[38] 寒武纪为其新一代人工智能处理器芯片采用Synopsys的HAPS产品[J].单片机与嵌入式系统应用,2018,18(06):96.

[39] 闫俊文.摩尔线程:成为中国的英伟达[J].中国企业家,2023,(07):70.

[40] 鲁蔚征,张峰,贺寅烜,等.华为昇腾神经网络加速器性能评测与优化[J].计算机学报,2022,45(08):1618-1637.

[41] 胡浩,沈莉,周清雷,等.基于LLVM编译器的节点融合优化方法[J].计算机科学,2020,47(S1):561-566+595.

[42] irong Z ,Xuejun Y ,Jianchun L , et al.Speculation-Free Function Table Construction in LLVM IR for Fine-Grained Control Flow Integrity[J].Journal of Circuits, Systems and Computers,2023,32(16).

[43] 桂中华. 基于MLIR的深度学习模型编译优化技术研究[D]. 河南:郑州大学,2024.

[44] 胡健,龚克,毛伊敏,等.基于Im2col的并行深度卷积神经网络优化算法[J].计算机应用研究,2022,39(10):2950-2956+2961.DOI:10.19734/j.issn.1001-3695.2022.03.0114.

[45] 姬晨晨,陈永青,韩孟之.基于国产加速器的三维卷积前向算子优化[J/OL].计算机工程,1-10[2025-03-02].https://doi.org/10.19678/j.issn.1000-3428.0068480.

[46] Krause M A ,Santos C P ,Lorenzon F A , et al.HBPB, applying reuse distance to improve cache efficiency proactively[J].Journal of Parallel and Distributed Computing,2024,191104919.

[47] Zuepke A ,Bastoni A ,Chen W , et al.MemPol: polling-based microsecond-scale per-core memory bandwidth regulation[J].Real-Time Systems,2024,60(3):369-412.

[48] Lenovo (Singapore) Pte. Ltd.; Patent Issued for PCI-E Real-Time Flow Control Optimization

万方数据

(USPTO 10,049,074)[J].Journal of Engineering,2018,5991.

[49] Yuan J ,Sun L ,Chu P , et al.Computing Unit and Data Migration Strategy under Limited Resources: Taking Train Operation Control System as an Example[J].Electronics,2024,13(21).

[50] Zheng C ,Zhejun L .Fixed global memory for controllable long text generation[J].Applied Intelligence,2022,53(11):13993-14007.

[51] Yarrabothu S R ,Telagathoti P.Comparative study of kernel algorithms on SIMD vector processor for 5G massive MIMO[J].International Journal of Advanced Intelligence Paradigms,2024,29(2-3):111-121.

[52] Lee C ,Oh J ,Ko H , et al.Monte Carlo simulation of interferometric measurement and wavefront shaping under influence of shot noise and camera noise[J].Journal of Physics: Photonics,2025,7(2):023001-023001..

[53] Yang X.The scaling-law flows: An attempt at scaling-law vector calculus[J]. Fractals, 2024, 32(4):2340126.

[54] Bajczi L ,Molnár V .Networcat: applying analysis techniques of shared memory software on message-passing distributed systems[J].Software and Systems Modeling,2025,(prepublish):1-20.

[55] Gholami M .SISO and PISO shift register in QCA technology and their application in the serial link[J].Heliyon,2025,11(4):e42873-e42873.

[56] Ruiz G C ,Acosta M ,Jorba O , et al.Optimized thread-block arrangement in a GPU implementation of a linear solver for atmospheric chemistry mechanisms[J].Computer Physics Communications,2024.

[57] Khan K H U A ,Al-Mouhamed M ,Fatayer A , et al.Padding Free Bank Conflict Resolution for CUDA-Based Matrix Transpose Algorithm[J].International Journal of Networked and Distributed Computing (IJNDC),2014,2(3):124-124.

[58] Han J ,Peng T ,Zhang X .A CUDA-based parallel optimization method for SM3 hash algorithm[J].The Journal of Supercomputing,2024,80(14):21431-21446.

[59] Correction to “Energy Efficient Resource Allocation in Cache - Enabled Fog Networks” [J].Transactions on Emerging Telecommunications Technologies,2025,36(2):e70072-e70072.

[60] Huzaifa M ,Alsop J ,Mahmoud A , et al.Inter-kernel Reuse-aware Thread Block Scheduling[J].ACM Transactions on Architecture and Code Optimization,2020,17(3):1-27.

[61] Park T .Random Partial Haar Wavelet Transformation for Single Instruction Multiple Threads[J].Journal of Digital Contents Society,2015,16(5):805-813.

[62] 孙万捷,高瞻,潘海燕,等.动态任务分配CUDA线程束步进体绘制[J].计算机辅助设计与图形学学报,2016,28(10):1630-1638.

万方数据

个人简历、在学期间发表的学术论文与研究成果

一、个人简历：

汤文浩，男，1999 年 6 月 29 日生，浙江省温州市人

2018 年 9 月—2022 年 7 月 宁波工程学院 网络工程 工学学士

2022 年 9 月—2025 年 7 月 郑州大学 计算机技术 工程硕士

二、在学期间发表的学术论文：

Research on Convolutional Operator Generation Based on MLIR[C], 2025 8th International Conference on Advanced Electronic Materials, Computers and Software Engineering.第一作者(已录用)

三、在学期间参加的科研项目：

[1] 国家重点研发计划，项目编号：2021YFB0300200

[2] 2022 年度河南省重大科技专项，项目编号：221100210600

万方数据

致谢

毕业之际，心中满怀感激。首先，衷心感谢母校郑州大学，为我提供了优越的学习环境和丰富的学术资源，使我能够在这三年中专注于学业与科研。母校的悉心栽培不仅让我汲取了宝贵的知识，还让我结识了一群志同道合的朋友，我们在求知的道路上携手同行，共同成长，收获了无数珍贵的回忆。

其次，诚挚感谢实验室的支持与帮助。实验室为我的研究工作提供了良好的条件，使我得以接触前沿领域的知识，拓宽了学术视野。同时，实验室提供的计算设备与资源支持，让我能够顺利开展科研工作，为学术研究贡献自己的力量。

最后，特别感谢我的导师、老师们以及同学们。在成长的道路上，导师和老师们给予了我悉心指导与耐心教诲，不仅传授专业知识，还教会了我如何为人处世，使我终生受益。在我遇到困难时，他们的鼓励与支持让我坚定信心，不断突破自我，勇敢前行。同学们的陪伴与相互扶持，也让这段求学旅程充满温暖与意义。

万方数据